ADAPTIVE MULTI-AGENT ORCHESTRATION FOR STOCKOUT RECOVERY AND RELEVANCE–DIVERSITY-AWARE PRODUCT DISCOVERY IN E-COMMERCE

A MINI PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY / BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted by

Student Name(s)	**[STUDENT NAME PLACEHOLDER — EDIT BEFORE SUBMISSION]**
Registration / Roll Number(s)	**[REGISTRATION NUMBER PLACEHOLDER — EDIT BEFORE SUBMISSION]**
Semester / Section	**[SEMESTER / SECTION PLACEHOLDER — EDIT BEFORE SUBMISSION]**

Under the Esteemed Guidance of

[FACULTY GUIDE NAME PLACEHOLDER, DESIGNATION]
[Department of Computer Science and Engineering]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
[INSTITUTION / UNIVERSITY NAME PLACEHOLDER]
[CITY, STATE, PIN CODE — PLACEHOLDER]

ACADEMIC YEAR: [2025 – 2026]
DATE OF SUBMISSION: [MONTH YEAR]

BONAFIDE CERTIFICATE

This is to certify that the mini-project report entitled "Adaptive Multi-Agent Orchestration for Stockout Recovery and Relevance–Diversity-Aware Product Discovery in E-Commerce" is a bonafide record of independent project work carried out by [STUDENT NAME PLACEHOLDER] (Registration No.: [REGISTRATION NUMBER PLACEHOLDER]) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology / Bachelor of Engineering in Computer Science and Engineering at [INSTITUTION / UNIVERSITY NAME PLACEHOLDER] during the academic year [2025–2026].

SIGNATURE OF PROJECT GUIDE

_____
[FACULTY GUIDE NAME PLACEHOLDER]
[Designation]
Department of CSE

SIGNATURE OF HEAD OF THE DEPARTMENT

_____
[HOD NAME PLACEHOLDER]
Head of the Department
Department of CSE

Submitted for the University / Viva-Voce Examination held on: _____

INTERNAL EXAMINER: _____

EXTERNAL EXAMINER: _____

ABSTRACT

Product discovery in modern e-commerce platforms faces a fundamental operational disconnect between information retrieval accuracy and real-time fulfillment feasibility. Conventional e-commerce search pipelines rank catalog items predominantly by lexical or semantic query similarity over internal inventory, or silently suppress out-of-stock products before presentation. When a shopper's highest-matching product experiences an internal stockout, standard search engines either fail to retrieve the desired item altogether or fill the top ranking slots with redundant, homogeneous alternatives from a single dominant brand. Furthermore, existing ranking pipelines rarely provide interpretable decision traces explaining why a specific fulfillment path—such as partner merchant sourcing or attribute-aligned substitution—was selected.

This mini-project presents a complete full-stack research prototype implementing Centralized Adaptive Orchestration for Stockout Recovery and Relevance–Diversity-Aware Product Discovery in E-Commerce. The backend architecture is built in Python using FastAPI, scikit-learn, pandas, NumPy, and SQLite, while the interactive web interface is developed in React 18 and Vite. The retrieval engine indexes 1,145 curated real-world product records derived from the Amazon Shopping Queries (ESCI) US dataset across four consumer electronics and appliance domains: laptops (319 SKUs), smartphones (369 SKUs), watches (228 SKUs), and refrigerators (229 SKUs). To enable controlled, repeatable experimentation without requiring proprietary warehouse telemetry, the data preparation pipeline synthesizes deterministic operational inventory states seeded via MD5 product hashes (854 internal in-stock items and 291 internal stockouts, of which 210 support simulated partner fulfillment).

The system implements and compares three experimental retrieval conditions: (1) a Baseline Engine executing TF-IDF unigram/bigram cosine similarity strictly over internal in-stock products; (2) an Optimized Engine incorporating regular-expression structured constraint parsing (price bounds, RAM, storage, screen size, color, brand, and delivery window), a weighted 70% semantic / 30% constraint relevance blend for general queries, hard price-bound filtering, and Intent-Adaptive Maximal Marginal Relevance (MMR) diversity reranking (using relevance weight 0.98 and brand penalty 0.01 for specific queries versus relevance weight 0.90 and brand penalty 0.04 for broad queries); and (3) an Orchestrated Engine that inspects the top-ranked candidate prior to internal stock filtering and dynamically routes execution across three explainable recovery paths:

internal fulfillment, exact-product partner recovery (when partner availability, price bounds, and delivery constraints are satisfied), and MMR-diversified substitute recovery.

Offline empirical evaluation across 173 real Amazon ESCI natural-language benchmark queries demonstrates that the Orchestrated pipeline matches the Baseline Hit@5 of 42.197% while improving Mean Reciprocal Rank (MRR) from 0.263 to 0.274 (+4.37%), raising top-5 Brand Diversity from 64.624% to 69.335% (+4.71 percentage points), and maintaining real-time responsiveness with an average latency of 31.49 ms per query (triggering stockout orchestration on 24.277% of queries). In a dedicated controlled stockout benchmark comprising 9 strict queries whose ground-truth exact targets are out of stock internally, Baseline and Optimized modes achieve 0.0% target recovery, whereas Orchestrated mode recovers 66.667% of all stockout cases (6/9) and 100.0% of partner-available stockout cases (6/6) at Rank #1 (Target MRR = 1.000). A 3-case substitute benchmark pilot additionally evaluates Amazon-labeled substitute retrieval (66.667% Substitute Hit@5). Beyond automated benchmarks, the platform provides a double-blind human-evaluation workflow enforcing a strict zero-target-leakage guarantee between candidate retrieval (POST /search) and post-retrieval evaluation (POST /evaluate), backed by SQLite session persistence and researcher CSV export. While the offline benchmarks and two internal verification database submissions are fully completed, formal large-scale external human-participant data collection remains planned as an ongoing study phase.

Keywords: Centralized Adaptive Orchestration, E-Commerce Product Discovery, Stockout Recovery, TF-IDF Semantic Retrieval, Maximal Marginal Relevance (MMR), Relevance–Diversity Trade-Off, Amazon ESCI Dataset, Double-Blind Human Evaluation.

TABLE OF CONTENTS

Note: The Table of Contents below is automatically generated and synchronized with document page numbers.

ABSTRACT 3

TABLE OF CONTENTS 5

CHAPTER 1 — INTRODUCTION 7

1.1 Project Overview 7

1.2 Problem Statement 8

1.3 Objectives 9

1.4 Scope of the Project 10

CHAPTER 2 — SYSTEM ANALYSIS 12

2.1 Existing System and Its Limitations 12

2.2 Proposed System and Its Advantages 13

2.3 Functional and Non-Functional Requirements 15

2.3.1 Functional Requirements 15

2.3.2 Non-Functional Requirements 16

CHAPTER 3 — SYSTEM DESIGN 17

3.1 System Architecture / Block Diagram 17

3.2 Workflow / Flowchart 18

3.2.1 Search Retrieval and Adaptive Stockout Orchestration Workflow 18

3.2.2 Double-Blind Human-Evaluation and Zero-Target-Leakage Workflow 20

3.3 Database Design / ER Diagram 21

3.4 Technologies and Tools Used 24

CHAPTER 4 — IMPLEMENTATION 25

4.1 Application Modules and Their Functions 25

4.2 Implementation Details 27

4.2.1 Semantic Retrieval 27

4.2.2 Structured Query Parsing 27
4.2.3 Relevance and Constraint Scoring 28
4.2.4 Intent-Adaptive Diversity Reranking 28
4.2.5 Adaptive Stockout Orchestration 29
4.2.6 API Endpoints 30
4.2.7 Human-Evaluation Workflow and Research Integrity Controls 30
4.3 Screenshots of the Application and Its Features 31
CHAPTER 5 — TESTING AND RESULTS 40
5.1 Test Cases and Results 40
5.2 Application Outputs and Decision Trace Inspection 41
5.3 Results and Discussion 42
5.3.1 Main ESCI Benchmark Across 173 Natural-Language Queries 42
5.3.2 Controlled Stockout Recovery Benchmark (9 Strict Cases) 45
5.3.3 Substitute Benchmark Pilot (3 Strict Cases) 47
5.3.4 Human-Evaluation Database Status 47
CHAPTER 6 — CONCLUSION AND FUTURE SCOPE 49
6.1 Conclusion 49
6.2 Limitations 49
6.3 Future Enhancements 50
REFERENCES 52
APPENDICES 54
Appendix A: Verified Repository Directory Structure 54
Appendix B: Sample Request and Response JSON Payloads 55
Appendix C: Dataset Preparation and Deterministic Inventory Synthesis 55
Appendix D: Installation, Execution, and Benchmark Reproduction Guide 56

CHAPTER 1 — INTRODUCTION

1.1 Project Overview

Modern electronic commerce (e-commerce) platforms serve as the primary interface through which consumers discover, compare, and purchase products across vast multi-category catalogs. At the core of every digital storefront lies an information retrieval and ranking pipeline tasked with translating free-form natural-language user queries into an ordered list of candidate items. Historically, e-commerce search engines have treated product retrieval primarily as a text-matching problem: given a user query, the engine scores catalog descriptions using lexical weighting schemes such as Term Frequency–Inverse Document Frequency (TF-IDF), BM25, or dense vector embeddings, and returns the highest-scoring items that are currently available in internal warehouse inventory [1], [2].

While pure relevance ranking performs adequately when catalog inventory is abundant and user queries are unambiguous, real-world retail environments regularly violate both assumptions. First, consumer queries span a wide spectrum of specificity, ranging from exact stock-keeping unit (SKU) lookups (for example, “2020 HP Pavilion Newest 14 Inch Laptop 10th Gen Intel i5 8GB RAM 256GB SSD Gold”) to open-ended, multi-constraint requirement queries (for example, “black gaming laptop under 80000 with 16gb ram and 512gb ssd”). When a user issues an exploratory or general query, ranking purely by lexical similarity frequently produces a top-k list dominated by near-duplicate variants from a single brand, starving the shopper of comparative breadth and brand diversity [3]. Conversely, if an engine aggressively enforces diversity on a highly specific query, it risks demoting the exact item the shopper intended to find.

Second—and critically for retail operations—high-demand products frequently experience temporary internal stockouts. Empirical retail studies consistently demonstrate that stockouts represent a major source of lost revenue, shopping cart abandonment, and diminished platform trust [4], [5]. When the highest-ranked product for a query has zero internal inventory (internal_stock = 0), a conventional internal-inventory-only search engine simply filters out the unavailable item prior to ranking and surfaces lower-relevance internal items without informing the user or attempting fulfillment recovery. In modern marketplace ecosystems, however, an out-of-stock internal item can often be fulfilled through an external partner merchant route (at a known partner price and

delivery lead time) or, when partner fulfillment is unavailable or violates user budget/delivery constraints, replaced with an attribute-aligned, brand-diversified substitute.

This mini-project, titled "Adaptive Multi-Agent Orchestration for Stockout Recovery and Relevance–Diversity-Aware Product Discovery in E-Commerce," designs, implements, and empirically evaluates an end-to-end web platform and retrieval engine that unifies semantic search, structured constraint parsing, intent-adaptive relevance–diversity reranking, and transaction-aware stockout recovery. Architecturally, it is important to characterize the implemented system accurately: the project implements Centralized Adaptive Orchestration within a unified search controller (SearchEngine in backend/search_engine.py), rather than a decentralized, peer-to-peer autonomous multi-agent network. Within this centralized orchestration architecture, specialized functional stages—query constraint parsing, semantic scoring, internal stock verification, partner-route feasibility validation, and Maximal Marginal Relevance (MMR) substitute reranking—are coordinated deterministically by a central controller that emits an explicit, human-readable decision_trace for every query.

1.2 Problem Statement

Conventional e-commerce search engines suffer from five interconnected algorithmic and architectural limitations when deployed in inventory-constrained environments:

- **1. Silent Failure Under Internal Stockouts:** When the highest-scoring catalog item matching a shopper's query is out of stock in internal inventory (internal_stock = 0), standard search pipelines pre-filter the catalog to internal_stock == 1 before ranking. Consequently, the exact target product is silently dropped, resulting in a 0% exact-target recovery rate under internal stockouts.
- **2. Inability to Validate Partner Fulfillment Feasibility:** Even when an identical product is obtainable through an affiliated partner merchant route (partner_available = 1), conventional search engines lack conditional routing logic to verify whether the partner offer satisfies the shopper's explicit price ceiling (partner_price <= max_price), price floor (partner_price >= min_price), and delivery lead-time requirements (partner_delivery_days <= max_delivery_days).
- **3. Suboptimal Substitute Selection:** When an out-of-stock product cannot be recovered via a partner route, returning raw TF-IDF neighbors from internal stock

ignores structured hardware/appliance specifications (such as RAM, storage capacity, screen size, color, and price bounds) and fails to incorporate feasible partner-fulfilled substitutes.

- **4. Static Relevance–Diversity Trade-Off:** Applying a static ranking objective across all queries creates an unavoidable trade-off: zero diversity reranking yields repetitive single-brand clusters on broad queries, whereas aggressive static diversification degrades exact-match precision (Hit@5 and MRR) on specific SKU queries.
- **5. Lack of Explainability and Experimental Isolation:** Black-box ranking pipelines do not expose structured diagnostic metadata explaining why a given retrieval path was chosen, nor do they separate candidate retrieval from target evaluation in interactive human studies, creating a risk of target leakage during experimental benchmarking.

Formal Problem Statement: Given a product catalog C where each item p in C possesses textual metadata, structured attributes, internal availability state s_int(p) in {0, 1}, and simulated partner availability state s_part(p) in {0, 1} with associated partner price and delivery latency, the problem is to design a centralized adaptive orchestration engine that: (a) parses natural-language constraints from user query q, (b) computes a hybrid semantic-constraint relevance score, (c) detects top-candidate internal stockouts in real time, (d) selects the optimal recovery action among Internal Fulfillment, Exact Partner Recovery, and Diversified Substitute Recovery subject to user constraints, (e) applies intent-adaptive MMR reranking tailored to query specificity, and (f) records an interpretable decision trace while supporting unbiased double-blind human evaluation.

1.3 Objectives

The specific engineering and research objectives realized in this project are as follows:

- **Objective 1 — Semantic Product Retrieval Over Real Catalog Data:** To construct a clean, multi-category e-commerce product catalog (1,145 items across laptops, phones, watches, and refrigerators) from the Amazon ESCI US dataset, augmented with deterministic, reproducible internal stockout and partner-fulfillment attributes.
- **Objective 2 — Systematic Multi-Condition Comparison:** To implement and benchmark three distinct search modes—Baseline (TF-IDF internal-only), Optimized (hybrid semantic + structured constraint scoring with MMR diversity), and

Orchestrated (adaptive stockout recovery + MMR diversity)—within a unified FastAPI backend.

- **Objective 3 — Intent-Adaptive Relevance–Diversity Reranking:** To develop an Intent-Adaptive Maximal Marginal Relevance (MMR) reranking algorithm that dynamically adjusts relevance weights and brand repetition penalties depending on whether the user's search intent is specific (relevance_weight = 0.98, brand_penalty = 0.01) or general (relevance_weight = 0.90, brand_penalty = 0.04).
- **Objective 4 — Adaptive Stockout Detection and Recovery:** To implement centralized adaptive stockout orchestration capable of inspecting the highest-ranked candidate product and automatically branching between internal fulfillment, exact partner recovery, and substitute recovery.
- **Objective 5 — Constraint-Checked Partner and Substitute Routing:** To validate exact partner recovery against explicit user budget and delivery-time constraints, and to synthesize a combined internal-and-partner substitute candidate pool when exact partner recovery is infeasible.
- **Objective 6 — Interpretable Orchestration Decision Traces:** To emit structured, human-readable orchestration decision traces (decision_trace) detailing stockout detection, candidate IDs, partner feasibility checks, and the selected action for researcher inspection.
- **Objective 7 — Interactive Discovery and Double-Blind Evaluation Website:** To build a full-stack React + FastAPI web application featuring interactive product discovery, a dynamic Product Specification Builder, a double-blind three-system comparison interface with a strict zero-target-leakage guarantee, and a researcher diagnostic dashboard.
- **Objective 8 — Persistent Experiment Logging and Reproducible Benchmarking:** To persist experimental sessions, task evaluations, and post-task questionnaire responses in a relational SQLite database (experiments.db) with automated CSV export for statistical analysis.

1.4 Scope of the Project

The scope of this mini-project encompasses data curation, retrieval algorithm design, centralized orchestration implementation, automated offline benchmarking, and web-based experimental infrastructure:

- **Catalog and Domain Scope:** The catalog covers four high-value consumer domains extracted from the Amazon ESCI US dataset: Laptops (319 products), Smartphones (369 products), Watches (228 products), and Refrigerators (229 products), totaling 1,145 verified products.
- **Controlled Operational Simulation Scope:** Because public academic datasets such as Amazon ESCI do not include live warehouse inventory counts, retail prices, or third-party partner shipping feeds, the data preparation script (backend/scripts/prepare_data.py) deterministically generates synthetic operational attributes (price, internal_stock, partner_available, partner_price, partner_delivery_days, rating, and structured hardware specs) seeded by MD5 hashes of each product's unique identifier.
- **Empirical Evaluation Scope:** Offline evaluation covers three distinct empirical suites: (i) a 173-query general and specific natural-language benchmark derived from Amazon ESCI ground-truth judgments, (ii) a 9-case strict controlled stockout recovery benchmark, and (iii) a 3-case strict Amazon-labeled substitute pilot benchmark.

Methodological Boundaries: In accordance with rigorous academic reporting standards, it is equally important to state what this system does not establish: (1) because inventory states, partner availability, partner pricing, and delivery lead times are deterministically simulated rather than connected to live enterprise ERP or merchant APIs, the benchmarks do not measure real-world third-party supply-chain reliability; (2) the study evaluates information retrieval quality (Hit@5, MRR, Brand Diversity, Constraint Match) and laboratory user perceptions, and does not establish actual monetary purchase conversion rates or long-term customer retention in a production retail deployment; and (3) while the human-evaluation platform and SQLite persistence pipeline are fully operational and verified via two internal test submissions, large-scale external human-participant cohort trials represent an ongoing extension.

CHAPTER 2 — SYSTEM ANALYSIS

2.1 Existing System and Its Limitations

In commercial e-commerce platforms and standard academic information retrieval baselines, product search is conventionally structured as a single-stage filter-then-rank pipeline over internal warehouse inventory. To establish a rigorous, reproducible reference point, it is necessary to distinguish between general commercial search systems discussed in the literature and the specific Baseline search condition implemented in this project (mode = 'baseline' in backend/search_engine.py).

In the general literature, traditional lexical search engines index product titles and descriptions using inverted indexes and score documents via TF-IDF or BM25. When inventory management is coupled to search, items with zero available stock are typically excluded via a hard boolean pre-filter (internal_stock == 1) before ranking so that shoppers are not shown unpurchasable listings. While large commercial marketplaces occasionally display 'More Buying Choices' or sponsored carousels on individual product detail pages, standard search result pages (SERPs) do not dynamically orchestrate partner-route validation or constraint-aware substitute reranking at the moment a top search hit encounters an internal stockout.

Within our experimental codebase, the implemented Baseline Mode reflects this conventional internal-inventory-only paradigm and exhibits five concrete technical characteristics verified directly from SearchEngine.baseline() in backend/search_engine.py:

- **1. Pure TF-IDF Semantic Retrieval:** The baseline engine vectorizes the user query using a pre-fitted TF-IDF vectorizer (up to 30,000 unigram and bigram features, English stop-words removed) and computes cosine similarity against the concatenated product_text field.
- **2. Strict Internal Inventory Filtering:** The candidate DataFrame is immediately restricted via candidates = candidates[candidates['internal_stock'] == 1]. Any product experiencing an internal stockout is unconditionally discarded prior to ranking.
- **3. Absence of Structured Constraint Scoring or Filtering:** Although parse_query(query) is executed to compute post-hoc diagnostic constraint_match badges for the UI, the baseline ranking score does not incorporate structured price

bounds, RAM, storage, screen size, color, brand, or delivery constraints, nor does it apply hard price filtering.

- **4. Absence of Diversity Reranking:** Candidates are sorted strictly in descending order of semantic_score. No Maximal Marginal Relevance (MMR) penalty or brand repetition penalty is applied, frequently causing top-5 results to contain multiple near-identical configurations from the same brand.
- **5. Zero Stockout Recovery Capability:** The baseline mode has no access to simulated partner inventory (partner_available) and performs no substitute orchestration when the best semantic match is out of stock internally.

2.2 Proposed System and Its Advantages

To overcome the limitations of conventional internal-only retrieval, the proposed system introduces a three-mode architecture within SearchLab: (1) Baseline, (2) Optimized, and (3) Orchestrated. By separating the algorithmic enhancements of constraint-aware diversity reranking (Optimized) from transaction-aware stockout recovery (Orchestrated), the experimental design isolates the exact empirical contribution of each mechanism. Table 2.1 summarizes the verified capabilities of the three experimental conditions against the source code in backend/search_engine.py.

Table 2.1: Verified Capability Comparison Across the Three Experimental Search Conditions

Capability / Mechanism	Baseline	Optimized	Orchestrated
Semantic relevance (TF-IDF unigram/bigram cosine)	Yes	Yes	Yes
Structured query constraint parsing & scoring (70/30 blend)	No	Yes	Yes
Hard price-bound candidate filtering (min_price / max_price)	No	Yes	Yes
Intent-adaptive MMR & brand diversity reranking	No	Yes	Yes
Internal inventory fulfillment (internal_stock == 1)	Yes	Yes	Yes
Exact-product partner stockout recovery (partner_available == 1)	No	No	Yes
Combined internal + partner substitute recovery	No	No	Yes
Decision trace (decision_trace)	Internal baseline behavior	Internal optimized behavior	Explainable orchestration

Based on verified benchmark executions across the 1,145-product catalog, the proposed system demonstrates several concrete, measured advantages without overstating retrieval performance:

- **1. Exact Target Recovery Under Internal Stockouts:** When the top-ranked candidate for a query is out of stock internally (which occurs in 42 out of 173 queries, or 24.28% of the main benchmark, and in 100% of the 9 controlled stockout cases), Baseline and Optimized modes fail to recover the exact target (0.0% recovery). The Orchestrated mode recovers 100.0% (6/6) of partner-available stockout targets at Rank #1, yielding an overall stockout recovery rate of 66.67% (6/9) and a Target MRR of 0.667 across the controlled stockout benchmark.
- **2. Preservation of Top-5 Accuracy with Higher Mean Reciprocal Rank:** On the 173-query ESCI benchmark, introducing unconstrained diversity reranking in Optimized mode slightly reduces Hit@5 from 42.20% (Baseline) to 41.04% because hard price filtering and diversity penalties occasionally displace a target product near the rank-5 boundary. However, the Orchestrated mode restores Hit@5 to 42.20% while elevating overall Mean Reciprocal Rank (MRR) from 0.263 (Baseline) and 0.262 (Optimized) to 0.274 (+4.37% over Baseline) by recovering stocked-out exact targets directly at Rank #1.
- **3. Significant Gain in Brand Diversity Without Sacrificing Relevance:** Through Intent-Adaptive MMR, top-5 Brand Diversity increases from 64.62% (3.231 unique brands per 5 results) in Baseline to 68.57% (3.382 unique brands) in Optimized and 69.34% (3.376 unique brands) in Orchestrated mode (+4.71 percentage points over Baseline), while Mean Relevance concurrently improves from 0.213 to 0.220.
- **4. Sub-35 Millisecond Real-Time Execution Latency:** By precomputing the candidate-to-candidate cosine similarity matrix once per MMR invocation and restricting candidate pools to the top 80 (Optimized) or top 100 (Orchestrated) items above 30% of the best relevance score, average query execution remains fast at 15.32 ms (Baseline), 33.28 ms (Optimized), and 31.49 ms (Orchestrated)—well below the 100 ms threshold for interactive human perception.

2.3 Functional and Non-Functional Requirements

2.3.1 Functional Requirements

Table 2.2 specifies the fifteen verified functional requirements (FR-01 through FR-15) implemented across the FastAPI backend and React frontend.

Table 2.2: Verified Functional Requirements of the Adaptive E-Commerce Orchestration Platform

Req. ID	Functional Requirement	Implemented Module & Verification Details
FR-01	Product Category Selection	WelcomePage.jsx & ProductDiscoveryPage.jsx allow filtering across all, laptop (319), phone (369), watch (228), and refrigerator (229).
FR-02	Natural-Language Search Input	Search bar accepts free-form specific SKU or general constraint queries with quick-fill example chips in Discovery mode.
FR-03	Search Condition Selection & Execution	Users/researchers can execute baseline, optimized, or orchestrated modes via POST /search or run all 3 in parallel.
FR-04	Structured Constraint Parsing	search_engine.py::parse_query() extracts min_price, max_price, ram_gb, storage_gb, screen_inches, color, brand, and max_delivery_days.
FR-05	Hybrid Product Ranking & Filtering	_optimized_relevance() blends 70% TF-IDF cosine similarity + 30% constraint match; _hard_filter() enforces price bounds.
FR-06	Real-Time Stockout Detection	Orchestrated mode inspects best = candidates.iloc[0] before internal stock filtering and flags internal_stock == 0.
FR-07	Partner-Route Feasibility Checks	Verifies partner_available == 1, partner_price within [min_price, max_price], and partner_delivery_days <= max_delivery_days.
FR-08	Substitute Recovery & MMR Reranking	Combines internal_substitutes and feasible partner_substitutes (excluding unavailable target) and applies Intent-Adaptive MMR.
FR-09	Search Result Presentation	Displays ranked cards with SKU, brand, price, fulfillment badge (Internal vs. Partner Recovered), delivery days, and score.
FR-10	Human-Evaluation Informed Consent	ExperimentWorkflowPage.jsx presents study protocol, requires explicit checkbox consent, and logs session via POST /experiments/session.
FR-11	Dynamic Product Specification Entry	ProductSpecificationBuilder.jsx lets participants build custom multi-attribute targets or choose predefined benchmark scenarios.
FR-12	Blinded Three-System Comparison	ComparisonPage.jsx randomizes Baseline, Optimized, and Orchestrated into System A/B/C and hides orchestration trace labels.
FR-13	Post-Task Questionnaire Submission	FeedbackPage.jsx captures target outcome, 9 perception scales, preferred condition, per-system ratings, and comments via POST /experiments/submit.
FR-14	Experiment Session & Task Logging	experiment_store.py logs every search trial in experiment_tasks and enforces single submission per session (HTTP 409 on duplicate).
FR-15	Researcher Dashboard & CSV Export	ResearcherPage.jsx displays cohort telemetry, live decision_trace inspector, and one-click CSV download via GET /experiments/export.csv.

2.3.2 Non-Functional Requirements

- **NFR-01 — Performance and Low Latency:** Search queries across the 1,145-item catalog must complete in under 50 ms on standard CPU hardware. Achieved: 15.32 ms (Baseline), 33.28 ms (Optimized), and 31.49 ms (Orchestrated).
- **NFR-02 — Reliability and Fault Tolerance:** The backend must gracefully validate all payloads via Pydantic schemas, enforce SQLite foreign-key/uniqueness integrity, return HTTP 409 Conflict on duplicate session submissions, and provide localStorage fallback if the API is temporarily unreachable.
- **NFR-03 — Usability:** The user interface must provide a clear step-by-step progress indicator during human evaluation, responsive card grids, accessible contrast, and immediate visual badges for stockout recovery and constraint satisfaction.
- **NFR-04 — Maintainability and Modularity:** Retrieval logic (search_engine.py), post-retrieval evaluation (evaluator.py), database persistence (experiment_store.py), and REST routing (api.py) must be strictly decoupled into modular Python components covered by automated tests.
- **NFR-05 — Participant Privacy and Anonymity:** No personally identifiable information (PII) such as names, emails, or IP addresses may be collected during human evaluation; sessions are tracked exclusively via anonymous random identifiers (SL-2026-XXXX).
- **NFR-06 — Research Integrity and Zero Target Leakage:** Hidden evaluation targets (target_product_id and target_constraints) must never be transmitted to POST /search, reference queries must never be auto-filled into participant query inputs, and condition identities must remain double-blinded during comparison.

CHAPTER 3 — SYSTEM DESIGN

3.1 System Architecture / Block Diagram

The SearchLab platform is engineered around a clean three-tier client–server architecture that enforces a strict architectural boundary between candidate retrieval (SearchEngine) and ground-truth target evaluation (Evaluator). Figure 3.1 illustrates the complete system architecture and component data flow.

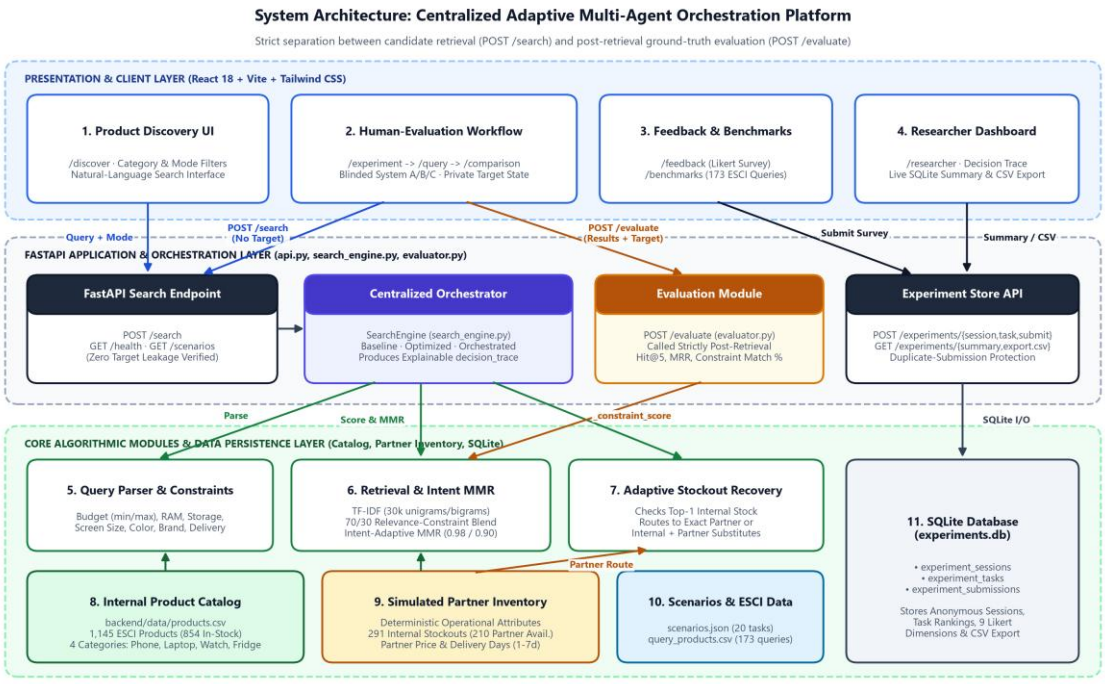


Figure 3.1: System Architecture and Data-Flow Block Diagram of the Adaptive E-Commerce Orchestration Platform

As depicted in Figure 3.1, the twelve architectural subsystems interact through well-defined interfaces:

- **1–2. User Interface & Frontend Application Layer:** Comprises the React 18 Single-Page Application (SPA) organized into five primary route workspaces: Overview (WelcomePage.jsx), Live Product Discovery (ProductDiscoveryPage.jsx), Controlled Human Evaluation (ExperimentWorkflowPage.jsx -> QueryPage.jsx -> ComparisonPage.jsx -> FeedbackPage.jsx), Benchmark Analytics (BenchmarksPage.jsx), and the Researcher Diagnostic View (ResearcherPage.jsx).
- **3. FastAPI Backend Service Layer:** Exposes ten RESTful endpoints via FastAPI (backend/api.py) with CORS middleware and Pydantic request validation, routing

incoming requests to the search engine, post-retrieval evaluator, or SQLite experiment store.

- **4–6. Search Engine, Query Parser, and Retrieval & Ranking Pipeline:** Encapsulates the TF-IDF vectorizer (30,000 features, 1–2 n-grams), regex-based structured constraint parser (parse_query), hybrid 70/30 relevance scorer (_optimized_relevance), hard price filter (_hard_filter), and Intent-Adaptive MMR reranker (_mmr).
- **7. Centralized Adaptive Orchestration Controller:** Inspects the highest-scoring candidate (best = candidates.iloc[0]) when mode == 'orchestrated', evaluates internal stock availability (internal_stock == 1), verifies partner fulfillment feasibility (partner_available == 1 and price/delivery constraints), and selects between internal_fulfillment, partner_exact_recovery, and substitute_recovery while recording an explainable decision_trace.
- **8–9. Internal Product Catalog & Simulated Partner Inventory:** Stored in backend/data/products.csv (1,145 curated Amazon ESCI US products), providing both internal stock states (854 in-stock, 291 stockout) and simulated partner fulfillment attributes (210 partner-available stockout items with partner price and delivery days).
- **10–12. Post-Retrieval Evaluation Module, SQLite Experiment Persistence, and Researcher Export:** The Evaluator (backend/evaluator.py) operates exclusively post-retrieval via POST /evaluate, receiving the already-retrieved product list alongside the participant's private hidden target to compute Hit@5, MRR, and constraint satisfaction. All session records, task logs, and questionnaire submissions are persisted in backend/data/experiments.db via ExperimentStore (backend/experiment_store.py) and surfaced to researchers via live summary telemetry and CSV export.

3.2 Workflow / Flowchart

3.2.1 Search Retrieval and Adaptive Stockout Orchestration Workflow

Figure 3.2 presents the detailed algorithmic flowchart governing query execution across the Baseline, Optimized, and Orchestrated search pipelines, followed by post-retrieval evaluation.

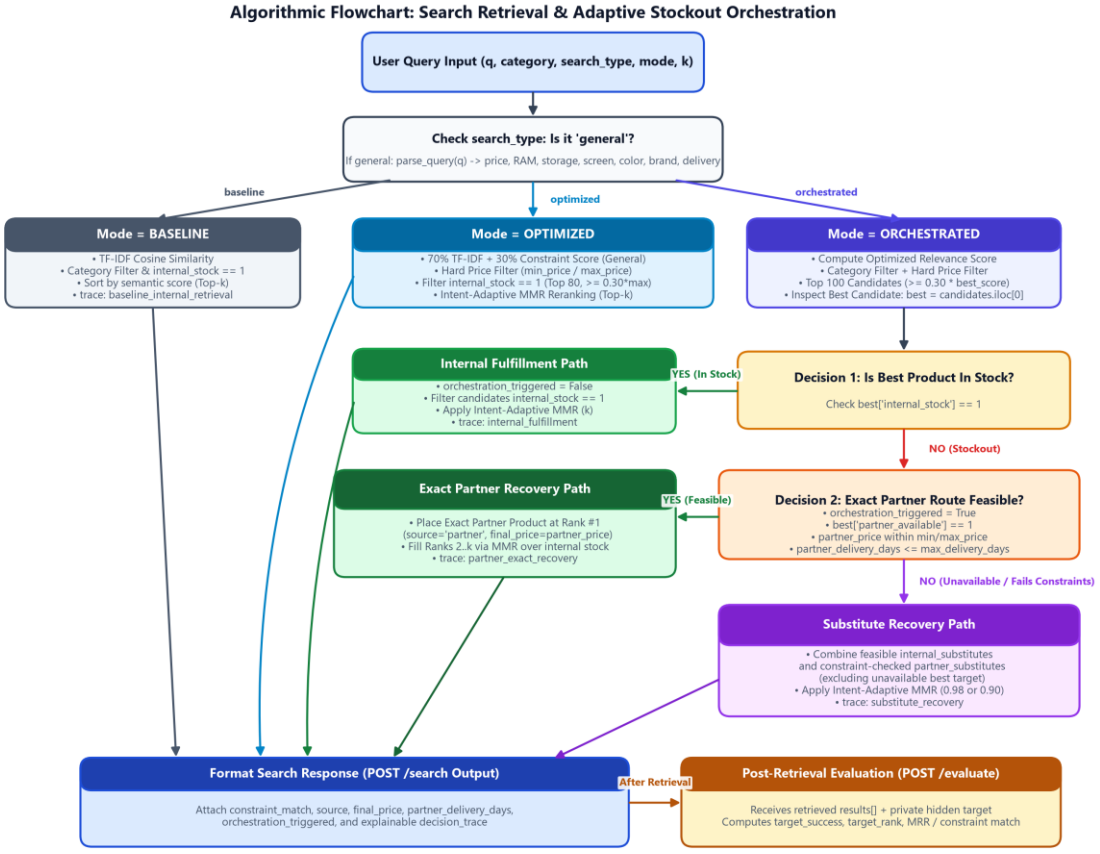


Figure 3.2: Algorithmic Flowchart for Multi-Mode Product Retrieval and Adaptive Stockout Orchestration

When a search request arrives at POST /search with parameters (query, mode, category, search_type, k), the engine first determines whether search_type == 'general'. For general searches, parse_query(query) extracts structured constraints; for specific SKU searches, constraints are bypassed so exact lexical matches are not penalized. Execution then branches by mode: Baseline filters immediately to internal_stock == 1 and sorts by raw TF-IDF similarity; Optimized computes the 70/30 hybrid relevance score, applies hard price bounds, filters to internal_stock == 1, truncates to the top 80 candidates scoring at least 30% of the maximum relevance, and applies Intent-Adaptive MMR; Orchestrated evaluates the top-100 candidate pool prior to internal stock filtering. If the top candidate (best) is in stock internally, Orchestrated executes the Internal Fulfillment path; if best is out of stock internally (internal_stock == 0), Orchestrated sets orchestration_triggered = True and checks whether exact partner recovery is feasible (partner_available == 1 and partner price/delivery within parsed constraints). If feasible, the exact product is recovered via the partner route at Rank #1 and remaining slots (2..k) are filled via MMR

over internal stock; otherwise, Orchestrated merges internal and constraint-valid partner substitutes (excluding best) and applies Intent-Adaptive MMR.

3.2.2 Double-Blind Human-Evaluation and Zero-Target-Leakage Workflow

A core methodological contribution of the system is its strict separation between candidate retrieval and hidden-target evaluation during human-subject experiments. Figure 3.3 illustrates the five-stage human-evaluation workflow and the client-side zero-leakage barrier.

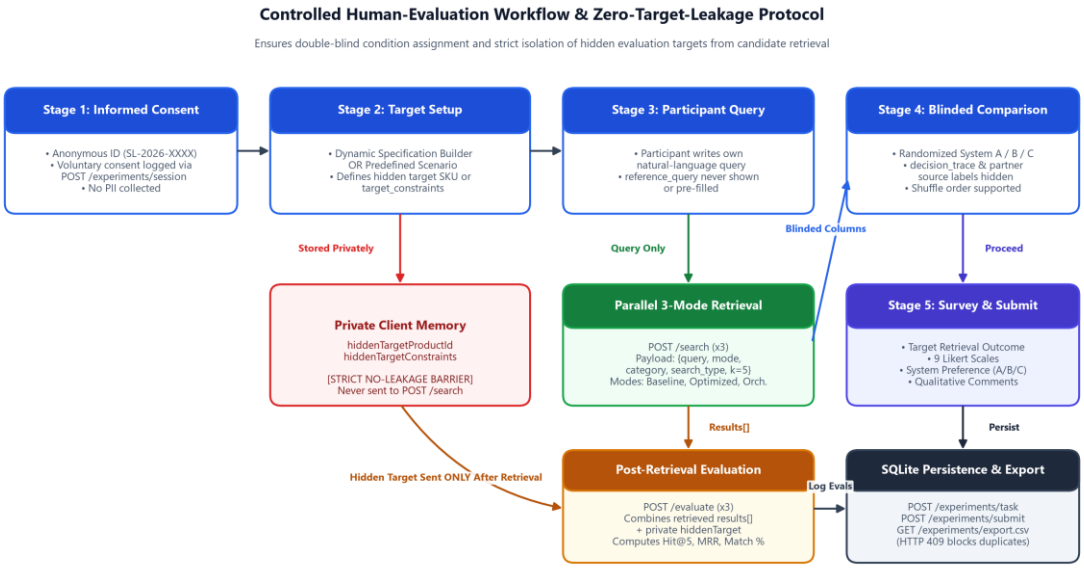


Figure 3.3: Controlled Human-Evaluation Workflow Demonstrating the Zero-Target-Leakage Protocol

As shown in Figure 3.3, the participant progresses through (1) Informed Consent, (2) Dynamic Target Specification or Scenario Selection, (3) Natural-Language Query Authoring (where reference queries are never displayed or pre-populated), (4) Double-Blind Three-System Comparison (where Baseline, Optimized, and Orchestrated are randomly assigned to System A, System B, and System C with orchestration traces hidden), and (5) Questionnaire Submission. Crucially, the hidden target (hiddenTargetProductId or hiddenTargetConstraints) is held exclusively in private client state during retrieval: POST /search receives only {query, mode, category, search_type, k}. Only after all three search modes have returned their ranked product lists does the frontend invoke POST /evaluate with the retrieved results and the private target.

3.3 Database Design / ER Diagram

The persistence layer for the human-evaluation study is implemented in SQLite3 (backend/data/experiments.db) and managed by the ExperimentStore class in backend/experiment_store.py. Inspection of the source code and SQLite schema confirms three relational tables: experiment_sessions, experiment_tasks, and experiment_submissions. Figure 3.4 depicts the Entity-Relationship (ER) diagram of the database.

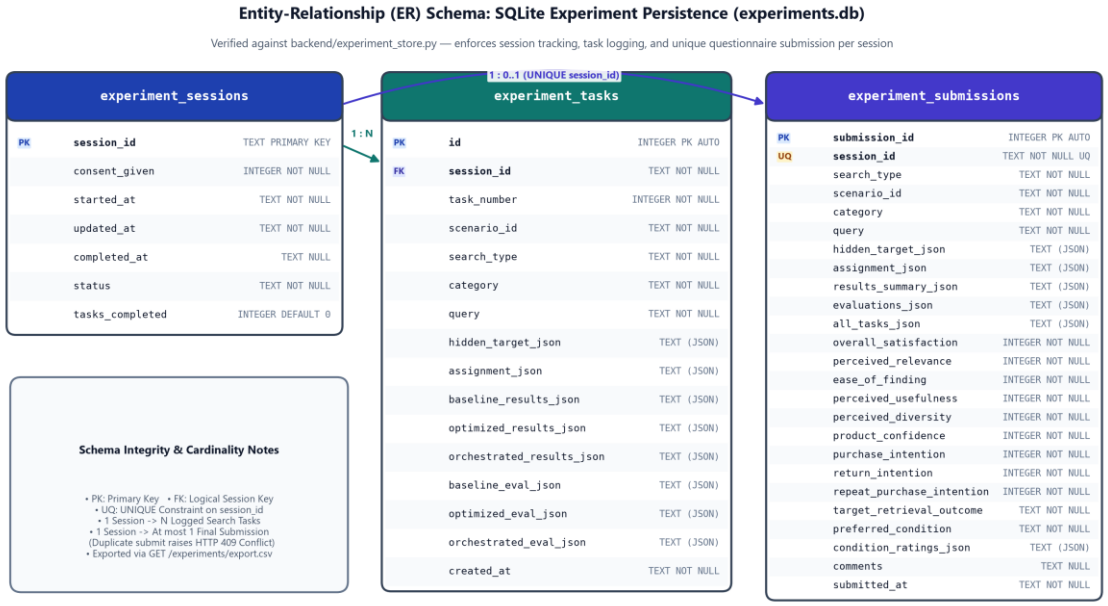


Figure 3.4: Entity-Relationship (ER) Schema of the SQLite Experiment Database (experiments.db)

Tables 3.1, 3.2, and 3.3 document the exact column definitions, data types, keys, and constraints verified from backend/experiment_store.py.

Table 3.1: Schema Definition for Table 'experiment_sessions'

Column Name	SQLite Data Type & Constraints	Key	Role / Description
session_id	TEXT PRIMARY KEY	PK	Unique anonymous session identifier (e.g., SL-2026-XXXX).
consent_given	INTEGER NOT NULL	-	Boolean integer flag (1 = explicit participant consent granted).
started_at	TEXT NOT NULL	-	UTC ISO-8601 timestamp when the session was initialized.
updated_at	TEXT NOT NULL	-	UTC ISO-8601 timestamp of the most recent session update.
completed_at	TEXT NULL	-	UTC ISO-8601 timestamp when the final questionnaire was submitted.

Column Name	SQLite Data Type & Constraints	Key	Role / Description
status	TEXT NOT NULL	-	Lifecycle state: 'in_progress' or 'completed'.
tasks_completed	INTEGER NOT NULL DEFAULT 0	-	Counter incremented each time a search task is logged via POST /experiments/task.

Table 3.2: Schema Definition for Table 'experiment_tasks'

Column Name	SQLite Data Type & Constraints	Key	Role / Description
id	INTEGER PRIMARY KEY AUTOINCREMENT	PK	Auto-incrementing surrogate primary key for each logged task.
session_id	TEXT NOT NULL	FK	Associates the task record with experiment_sessions.session_id (1:N).
task_number	INTEGER NOT NULL	-	Sequential 1-based index of the task within the participant's session.
scenario_id	TEXT NOT NULL	-	Identifier of the selected scenario or dynamic specification builder task.
search_type	TEXT NOT NULL	-	Query intent mode: 'specific' or 'general'.
category	TEXT NOT NULL	-	Product domain ('laptop', 'phone', 'watch', or 'refrigerator').
query	TEXT NOT NULL	-	Exact natural-language query authored by the participant.
hidden_target_json	TEXT NOT NULL	-	JSON-serialized hidden target SKU/metadata or structured target constraints.
assignment_json	TEXT NOT NULL	-	JSON mapping of blinded columns {'A': mode, 'B': mode, 'C': mode}.
baseline_results_json	TEXT NOT NULL	-	JSON array of top-k products returned by Baseline mode.
optimized_results_json	TEXT NOT NULL	-	JSON array of top-k products returned by Optimized mode.
orchestrated_results_json	TEXT NOT NULL	-	JSON array of top-k products returned by Orchestrated mode.
baseline_eval_json	TEXT NOT NULL	-	JSON post-retrieval evaluation metrics for Baseline mode.
optimized_eval_json	TEXT NOT NULL	-	JSON post-retrieval evaluation metrics for Optimized mode.
orchestrated_eval_json	TEXT NOT NULL	-	JSON post-retrieval evaluation metrics for Orchestrated mode.
created_at	TEXT NOT NULL	-	UTC ISO-8601 timestamp when the task was recorded.

Table 3.3: Schema Definition for Table 'experiment_submissions'

Column Name	SQLite Data Type & Constraints	Key	Role / Description
submission_id	INTEGER PRIMARY KEY AUTOINCREMENT	PK	Unique auto-incrementing identifier for each completed questionnaire.
session_id	TEXT NOT NULL UNIQUE	FK, UQ	Links to experiment sessions.session id; UNIQUE enforces at most 1 submission per session.
search_type	TEXT NOT NULL	-	Search type ('specific' or 'general') of the final evaluated task.
scenario_id	TEXT NOT NULL	-	Scenario or custom builder identifier of the evaluated task.
category	TEXT NOT NULL	-	Product category evaluated in the session.
query	TEXT NOT NULL	-	Participant-authored natural-language query.
hidden_target_json	TEXT NOT NULL	-	JSON representation of the hidden target item or constraints.
assignment_json	TEXT NOT NULL	-	JSON blinded system assignment map (System A/B/C -> mode).
results_summary_json	TEXT NOT NULL	-	JSON summary of retrieved results across all three modes.
evaluations_json	TEXT NOT NULL	-	JSON post-retrieval evaluation outcomes for all three modes.
all_tasks_json	TEXT NOT NULL	-	JSON array archiving all tasks performed during the session.
overall_satisfaction	INTEGER NOT NULL	-	Participant rating for Overall Satisfaction (validated 1..7 in API).
perceived_relevance	INTEGER NOT NULL	-	Participant rating for Perceived Relevance.
ease_of_finding	INTEGER NOT NULL	-	Participant rating for Ease of Finding.
perceived_usefulness	INTEGER NOT NULL	-	Participant rating for Perceived Usefulness.
perceived_diversity	INTEGER NOT NULL	-	Participant rating for Perceived Diversity.
product_confidence	INTEGER NOT NULL	-	Participant rating for Product Confidence.
purchase_intention	INTEGER NOT NULL	-	Participant rating for Purchase Intention.
return_intention	INTEGER NOT NULL	-	Participant rating for Return Intention.
repeat_purchase_intention	INTEGER NOT NULL	-	Participant rating for Repeat Purchase Intention.
target_retrieval_outcome	TEXT NOT NULL	-	Self-reported outcome ('exact', 'alt', or 'none' / descriptive text).
preferred_condition	TEXT NOT NULL	-	De-blinded preferred condition ('baseline', 'optimized', 'orchestrated', 'none').
condition_ratings_json	TEXT NOT NULL	-	JSON dictionary of per-system ratings.
comments	TEXT NULL	-	Optional open-ended qualitative feedback from the participant.
submitted_at	TEXT NOT NULL	-	UTC ISO-8601 submission timestamp.

3.4 Technologies and Tools Used

Table 3.4 summarizes the verified software technologies, libraries, and versions used across the backend, frontend, data pipeline, and testing infrastructure.

Table 3.4: Verified Technology Stack and Third-Party Libraries

Layer / Domain	Technology / Library	Verified Version / Spec	Primary Role in Project
Backend Language	Python	Python 3.12+	Core implementation language for search, orchestration, evaluation, and scripts.
Backend Web API	FastAPI & Uvicorn	fastapi >= 0.115.0, uvicorn >= 0.30.0	High-performance asynchronous REST API server and OpenAPI schema generation.
Schema Validation	Pydantic	pydantic >= 2.8.0	Strict request/response payload validation and type enforcement in api.py.
Machine Learning / IR	scikit-learn	scikit-learn >= 1.5.0	TfidfVectorizer (30,000 unigram/bigram features) and cosine_similarity computation.
Data Processing	pandas & NumPy	pandas >= 2.2.0, numpy >= 1.26.0	In-memory catalog DataFrame manipulation, vectorized scoring, and CSV aggregation.
Dataset Ingestion	PyArrow	pyarrow >= 16.0.0	Reading raw Amazon ESCI Parquet shards during prepare_data.py execution.
Plotting & Figures	matplotlib	matplotlib >= 3.9.0	Generation of publication-grade benchmark charts in generate_plots.py.
Database Engine	SQLite3	Python stdlib sqlite3	Embedded relational storage (backend/data/experiments.db) for human-eval logs.
Frontend Framework	React & React DOM	react ^18.3.1	Component-based Single-Page Application UI and state management.
Build Tool & Bundler	Vite	vite ^5.4.8	Fast ES-module frontend development server (port 5173) and production bundler.
UI Iconography	Lucide React	lucide-react ^0.454.0	Clean vector icons for product categories, status badges, and diagnostic controls.
Automated Testing	Node.js Test Runner & Python	node:test / node:assert, pytest	Automated unit/integration tests in frontend/tests/searchlab.test.js and backend/tests/.

CHAPTER 4 — IMPLEMENTATION

4.1 Application Modules and Their Functions

The codebase is organized into two primary subsystems inside d:\DAA\adaptive-ecommerce-orchestration: the Python backend (backend/) and the React 18 frontend (frontend/). Table 4.1 documents every verified source module across both directories.

Table 4.1: Verified Backend and Frontend Source Modules and Their Functional Responsibilities

Subsystem	File Path	Primary Classes / Functions & Functional Role
Backend Core	backend/api.py	FastAPI app instance, Pydantic request/response schemas, CORS setup, and 10 REST endpoints (/health, /scenarios, /targets, /search, /evaluate, /experiments/*).
Backend Core	backend/search_engine.py	SearchEngine class: TF-IDF indexing, parse_query(), _constraint_score(), optimized relevance(), hard filter(), _mmr(), baseline(), optimized(), orchestrated().
Backend Core	backend/evaluator.py	Evaluator class: evaluate results(), evaluate_specific_target() (Hit@5, rank, MRR), and evaluate_general_constraints() (attribute match ratio, diversity).
Backend Core	backend/experiment_store.py	ExperimentStore class: SQLite DDL initialization, start_session(), log_task(), submit feedback() (with HTTP 409 uniqueness check), summary(), export csv().
Data Scripts	backend/scripts/prepare_data.py	Extracts Amazon ESCI US data, maps 4 categories, removes accessories, and deterministically synthesizes price, internal_stock, partner_available, and hardware specs.
Data Scripts	backend/scripts/generate_scenarios.py	Generates 8 curated human-evaluation scenarios (4 specific target SKUs, 4 general constraint tasks) saved to backend/data/experiment scenarios.jso n.
Data Scripts	backend/scripts/find_recovery_case.py	Diagnostic utility that scans benchmark queries to identify stocked-out exact targets recoverable via partner routing (e.g., B083CYM3TZ).
Benchmarks	backend/benchmarks/benchmark.py	Executes the 173-query ESCI benchmark across Baseline, Optimized, and Orchestrated modes; outputs benchmark_results.csv, benchmark_summary.csv, benchmark_by_category.csv.
Benchmarks	backend/benchmarks/benchmark_stockouts.py	Executes the 9-case strict controlled stockout benchmark where ground-truth exact targets have internal_stock == 0; outputs stockout_benchmark_*.csv.

Subsystem	File Path	Primary Classes / Functions & Functional Role
Benchmarks	backend/benchmarks/benchmark_substitutes.py	Executes the 3-case strict substitute pilot using Amazon ESCI 'S' (Substitute) judgments; outputs substitute_benchmark_*.csv.
Benchmarks	backend/benchmarks/generate_plots.py	Generates publication-quality matplotlib bar charts in backend/results/ (search quality diversity.png, latency_comparison.png, stockout_recovery.png).
Backend Tests	backend/tests/test_demo.py, test_search.py, test_general.py	Verification scripts testing stockout partner recovery (B083CYM3TZ), mode-by-mode ranking differences, and general constraint parsing.
Frontend Core	frontend/src/app/routes.jsx & App.jsx	Defines top-level navigation state and routes between Welcome, ProductDiscovery, ExperimentWorkflow, Benchmarks, and Researcher views.
Frontend Core	frontend/src/services/api.js	REST client wrapper with 12-second AbortController timeouts, payload builders (buildSearchRequestPayload, buildEvaluationPayload), and local fallback.
Frontend Core	frontend/src/hooks/useExperimentSession.js	Custom React hook managing anonymous session ID, consent state, dynamic/scenario targets, blinded assignment (System A/B/C), and zero-leakage state.
Frontend UI	frontend/src/pages/WelcomePage.jsx & ProductDiscoveryPage.jsx	Landing overview with category cards and interactive live search explorer with mode/intent toggles, example query chips, and product cards.
Frontend UI	frontend/src/components/experiment/ProductSpecificationBuilder.jsx	Interactive form allowing human-study participants to define custom target specifications (category, brand, budget, RAM, storage, screen, color, delivery).
Frontend UI	frontend/src/pages/ExperimentWorkflowPage.jsx, QueryPage.jsx, ComparisonPage.jsx, FeedbackPage.jsx	Four-stage controlled human evaluation UI: consent & target setup -> query entry -> double-blind 3-column comparison -> post-task survey.
Frontend UI	frontend/src/pages/BenchmarksPage.jsx & ResearcherPage.jsx	Visualizes offline benchmark tables/charts and provides live SQLite cohort diagnostics, orchestration decision trace inspection, and CSV export.
Frontend Tests	frontend/tests/searchlab.test.js	12 automated Node.js unit and integration tests verifying zero target leakage, blinded assignment randomization, and API payload schemas.

4.2 Implementation Details

4.2.1 Semantic Retrieval

Upon initialization, SearchEngine (backend/search_engine.py) loads backend/data/products.csv (1,145 rows) into a pandas DataFrame. For every product p, a unified textual representation product_text is constructed during data preparation by concatenating five catalog fields: title, brand, color, bullets, and description. The engine fits a scikit-learn TfidfVectorizer configured with stop_words='english', max_features=30000, and ngram_range=(1, 2):

Listing 4.1: TF-IDF Vectorizer Initialization in backend/search_engine.py

```
self.vectorizer = TfidfVectorizer(
    stop_words="english",
    max_features=30000,
    ngram_range=(1, 2),
)
self.tfidf_matrix = self.vectorizer.fit_transform(self.df["product_text"])
```

Given an incoming natural-language query q, _semantic_scores(q) transforms q into the same 30,000-dimensional unigram/bigram vector space and computes cosine similarity against all catalog items:

$$s_sem(q, p) = cos(v_q, v_p) = (v_q \cdot v_p) / (||v_q||_2 \, ||v_p||_2) \quad \textbf{(4.1)}$$

4.2.2 Structured Query Parsing

When a user executes a broad requirement search (search_type == 'general'), SearchEngine.parse_query(query) extracts eight structured constraint dimensions using deterministic regular expressions and catalog brand lookup:

- **1–2. Price Bounds (min_price, max_price):** Detects range expressions (between X and Y or X to Y), upper-bound expressions (under X, below X, less than X, max X, budget X, <= X), and lower-bound expressions (above X, over X, more than X, min X, >= X). Numeric multipliers for 'k' (x 1,000) and 'lakh'/'lac' (x 100,000) as well as ₹ symbols and commas are normalized automatically.
- **3–4. Memory and Storage Specifications (ram_gb, storage_gb):** Extracts RAM capacity via r'(\d+)\s*gb\s*ram' and storage capacity via r'(\d+)\s*(gb|tb)\s*(?:ssd|hdd|storage|rom|internal)?', converting terabytes (tb) to gigabytes by multiplying by 1,024. If 'gb' appears without an explicit 'ram' keyword, values <= 24 GB are inferred as RAM and values > 24 GB are inferred as storage.

- **5–6. Display Size (screen_inches) and Color (color):** Extracts display diagonal size via r'(\d+(?:\.\d+)?)\s*(?:inch|inches|")' and matches twelve canonical colors (black, white, silver, gray, grey, blue, red, gold, green, pink, purple, rose gold), normalizing 'grey' to 'gray'.
- **7–8. Brand (brand) and Delivery Window (max_delivery_days):** Scans the lowercased query against all unique catalog brands (self.known_brands) using word-boundary regexes r'\b' + re.escape(brand) + r'\b', and extracts delivery lead-time bounds via r'(?:within|under|in)\s*(\d+)\s*days?'.

4.2.3 Relevance and Constraint Scoring

For each candidate item p and parsed constraint dictionary C_q, _constraint_score(row, constraints, use_partner=False) evaluates satisfaction across all active constraints in C_q. Price, RAM (row['ram_gb'] >= ram_gb), storage (row['storage_gb'] >= storage_gb), color, brand, and delivery window (delivery_days <= max_delivery_days) are scored as binary indicators in {0.0, 1.0}, while screen size receives 1.0 if |row['screen_inches'] - screen_inches| <= 1.2 inches and 0.5 if within 2.5 inches. The item's overall constraint match score s_con(q, p) is the arithmetic mean of all matched constraint indicators.

In Optimized and Orchestrated modes, _optimized_relevance() computes a hybrid relevance score. If search_type == 'specific' or no structured constraints are present in the query, relevance equals pure semantic similarity; otherwise, it blends 70% semantic similarity with 30% structured constraint alignment:

$$R_opt(q, p) = 0.70 \cdot s_sem(q, p) + 0.30 \cdot s_con(q, p) \quad \textit{(for general queries with constraints)} \qquad \textbf{(4.2)}$$

Subsequently, _hard_filter() enforces strict price bounds (price <= max_price and price >= min_price) for general queries whenever doing so leaves at least one candidate in the pool. To keep subsequent diversity reranking fast and prevent low-relevance noise from entering the top-k slots, Optimized mode retains the top 80 candidates (and Orchestrated retains the top 100 candidates) whose relevance score is at least 30% of the top candidate's score (relevance_score >= 0.30 * best_score).

4.2.4 Intent-Adaptive Diversity Reranking

To balance ranking precision on specific SKU lookups against brand exploration on broad requirement queries, SearchEngine._mmr_settings(search_type) implements Intent-Adaptive Maximal Marginal Relevance (MMR):

Listing 4.2: Intent-Adaptive MMR Hyperparameters in backend/search_engine.py

```
def _mmr_settings(self, search_type: str) -> Tuple[float, float]:
    # Specific product search should stay very close to pure relevance.
    # General requirement search benefits from gentle brand/item diversification.
    if search_type == "specific":
        return 0.98, 0.01
    return 0.90, 0.04
```

Given candidate pool D, already-selected set S, relevance weight lambda in {0.98, 0.90}, and brand penalty beta in {0.01, 0.04}, the MMR objective at each greedy selection step chooses candidate p in D \ S maximizing:

$$MMR(p) = lambda \cdot R_opt(q, p) - (1 - lambda) \cdot max_\{s \text{ in } S\} cos(v_p, v_s) - beta \cdot I(brand(p) \text{ in } brands(S)) \quad \textbf{(4.3)}$$

where cos(v_p, v_s) is looked up from a precomputed candidate-to-candidate cosine similarity matrix (sim_matrix = cosine_similarity(sub_tfidf, sub_tfidf)) computed once per _mmr() call, ensuring O(k · |D|) selection overhead.

4.2.5 Adaptive Stockout Orchestration

In SearchEngine.orchestrated(), the controller ranks the category-filtered and price-filtered candidate pool (up to 100 items) by R_opt(q, p) before filtering by internal stock. Let p* = candidates.iloc[0] denote the highest-ranked candidate. The centralized orchestrator executes a three-way conditional decision tree:

- **Path 1 — Internal Fulfillment (p*['internal_stock'] == 1):** The orchestrator sets orchestration_triggered = False, filters candidates to internal_stock == 1, applies Intent-Adaptive MMR to select the top-k internal products, and records selected_action = 'internal_fulfillment' in decision_trace.
- **Path 2 — Exact Partner Recovery (p*['internal_stock'] == 0 and Partner Route Feasible):** When p*['internal_stock'] == 0, the orchestrator sets orchestration_triggered = True and verifies three partner feasibility predicates: (i) p*['partner_available'] == 1, (ii) p*['partner_price'] is within [min_price, max_price] (if specified), and (iii) p*['partner_delivery_days'] <= max_delivery_days (if specified). When all predicates hold, p* is recovered at Rank #1 with source = 'partner' and final_price = partner_price, while Ranks 2..k are populated via Intent-Adaptive MMR over in-stock internal alternatives. The trace records selected_action = 'partner_exact_recovery'.
- **Path 3 — Substitute Recovery (p*['internal_stock'] == 0 and Partner Route Infeasible):** When p*['internal_stock'] == 0 and the exact partner route is unavailable

or violates user price/delivery constraints, the orchestrator constructs a unified substitute pool combining internal_substitutes (internal_stock == 1, excluding p*) and constraint-valid partner_substitutes (internal_stock == 0, partner_available == 1, satisfying partner price and delivery bounds, excluding p*). Partner substitutes are rescored using their partner price/delivery attributes, and the combined pool is reranked via Intent-Adaptive MMR to return the top-k substitutes with selected_action = 'substitute_recovery'.

4.2.6 API Endpoints

Table 4.2 documents all ten REST endpoints implemented in backend/api.py.

Table 4.2: Complete FastAPI Backend Endpoint Reference (backend/api.py)

Method	Endpoint Path	Request Model / Params	Response Payload & Purpose
GET	/health	None	Returns {status: 'ok', catalog_size: 1145}.
GET	/scenarios	None	Returns the 8 curated evaluation scenarios from experiment_scenarios.json.
GET	/targets	category: Optional[str], limit: int = 24	Returns candidate target products for human-evaluation setup.
POST	/search	SearchRequest {query, mode, category, search_type, k}	Executes baseline, optimized, or orchestrated search; returns results[], latency_ms, decision_trace.
POST	/evaluate	EvaluateRequest {results, search_type, target_product_id, target_constraints}	Post-retrieval evaluation only; computes Hit@5, target_rank, MRR, constraint match, and brand diversity.
POST	/experiments/session	ExperimentSessionRequest {session_id, consent_given}	Registers or updates an anonymous human-evaluation session in SQLite.
POST	/experiments/task	ExperimentTaskRequest {session_id, task_number, ...}	Logs a completed 3-system search comparison trial into experiment_tasks.
POST	/experiments/submit	ExperimentSubmitRequest {session_id, ratings, ...}	Persists final questionnaire in experiment_submissions; raises HTTP 409 on duplicate session_id.
GET	/experiments/summary	None	Aggregates live SQLite cohort counts, Likert means, preference distribution, and recent submissions.
GET	/experiments/export.csv	None	Streams all rows from experiment_submissions as a downloadable CSV file.

4.2.7 Human-Evaluation Workflow and Research Integrity Controls

The human-evaluation workflow (managed by useExperimentSession.js and rendered across ExperimentWorkflowPage.jsx, QueryPage.jsx, ComparisonPage.jsx, and

FeedbackPage.jsx) enforces three strict methodological safeguards verified by automated unit tests in frontend/tests/searchlab.test.js:

- **1. Zero Target Leakage During Retrieval:** In frontend/src/services/api.js, buildSearchRequestPayload() explicitly constructs the POST /search body with only five keys: {query, mode, category, search_type, k}. Hidden target identifiers (target_product_id) and target constraint objects (target_constraints) are stripped and never transmitted during retrieval.
- **2. Authentic Participant Query Authoring:** In QueryPage.jsx, participants view the target product's attributes or their custom specification summary, but must author their own natural-language search query from scratch; reference queries are never pre-filled.
- **3. Double-Blind Presentation & Questionnaire Scale Note:** ComparisonPage.jsx maps the three search modes onto System A, System B, and System C using a randomized permutation per session and suppresses internal/partner source badges and orchestration traces until the post-task survey in FeedbackPage.jsx is submitted. (Note on questionnaire scale implementation: FeedbackPage.jsx renders 5-point Likert rating buttons 1..5 in the participant UI for compact visual ergonomics, while the backend Pydantic validator in api.py and summary aggregator in ResearcherPage.jsx support a 1..7 integer range.)

4.3 Screenshots of the Application and Its Features

To document the actual implemented user interface and researcher tools, twelve genuine high-resolution screenshots were captured directly from the live running application (FastAPI backend on port 8000 and React/Vite frontend on port 5173) and are presented in Figures 4.1 through 4.12.

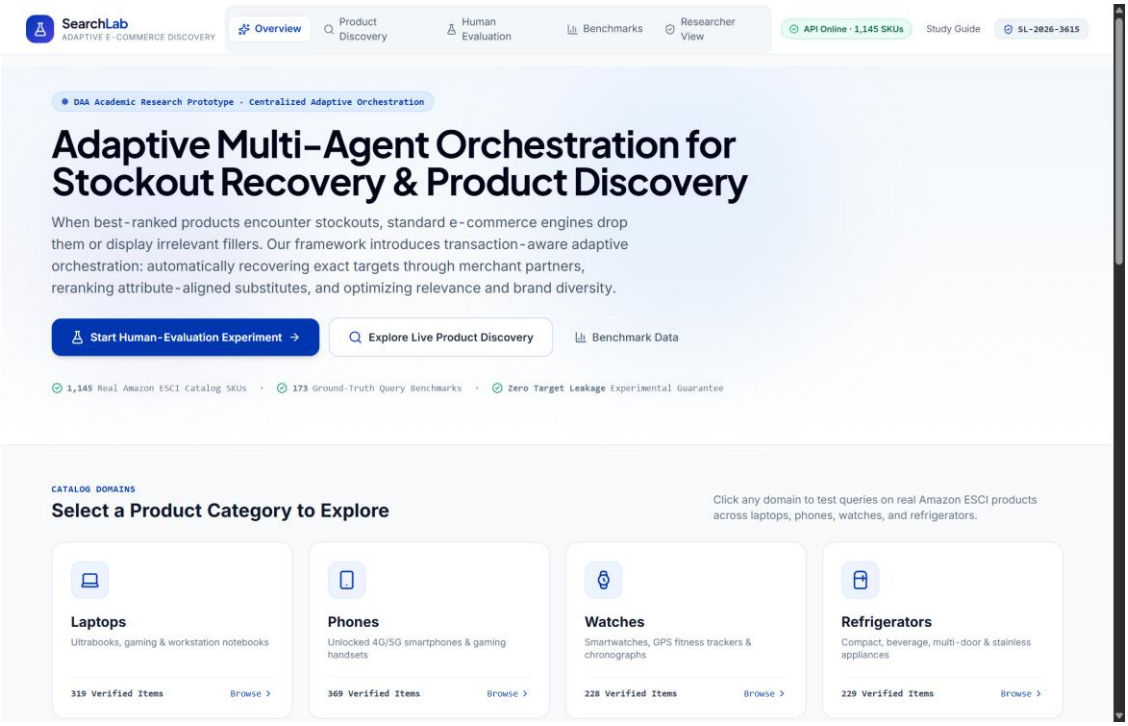


Figure 4.1: Application Landing Page and Hero Overview (WelcomePage.jsx)

Displays the project title, architectural summary, live API status indicator (1,145 SKUs online), anonymous session badge, and navigation links to Live Product Discovery and the Controlled Human-Evaluation Experiment.

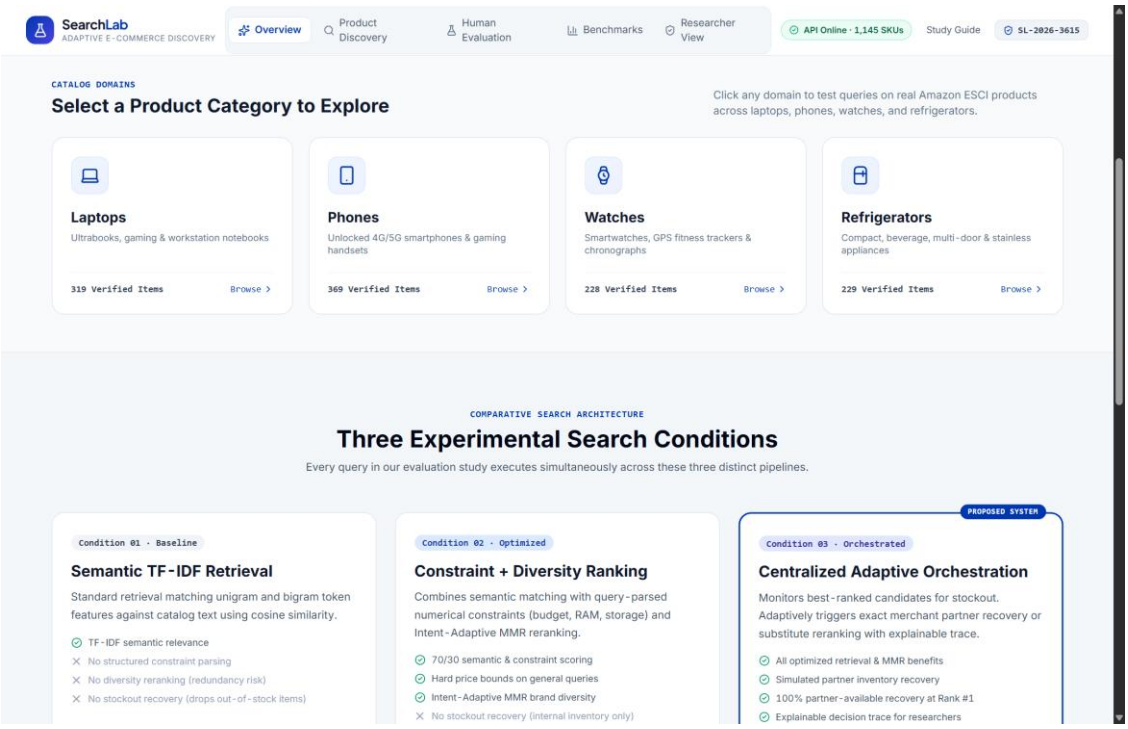


Figure 4.2: Interactive Catalog Domain Selection Cards and Three-Mode Architecture Matrix (WelcomePage.jsx)

Illustrates the four real Amazon ESCI catalog domains—Laptops (319 items), Phones (369 items), Watches (228 items), and Refrigerators (229 items)—followed by the architectural comparison of Baseline, Optimized, and Orchestrated modes.

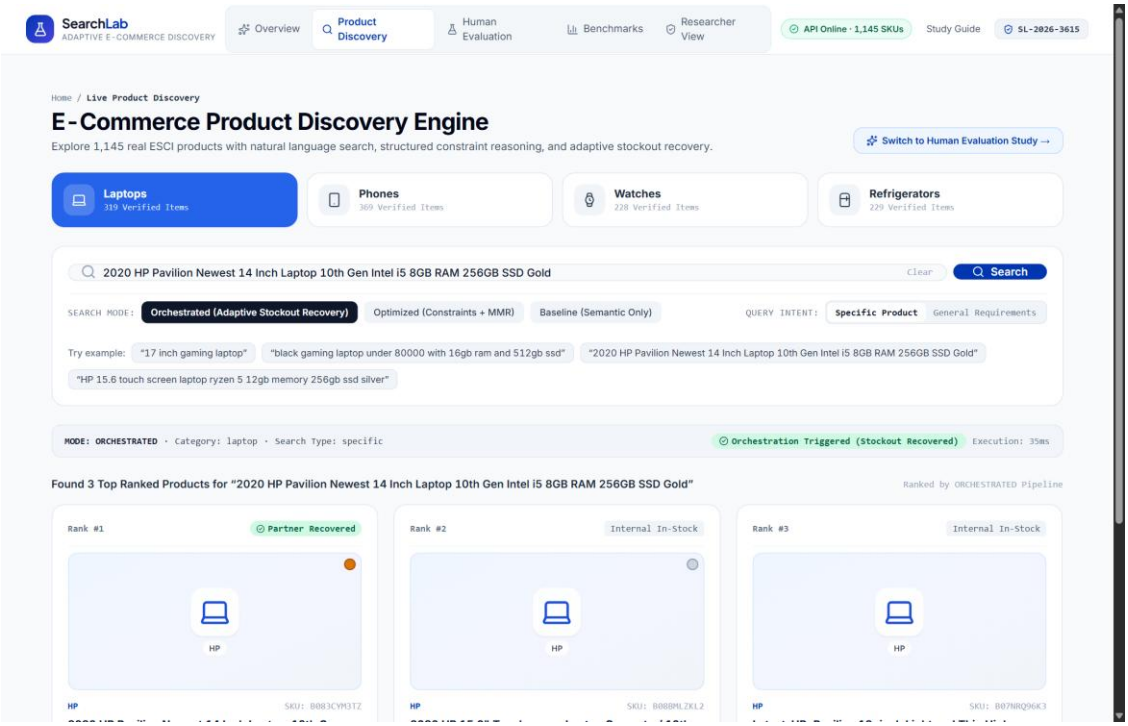


Figure 4.3: Live Product Discovery Interface and Search Configuration Controls (ProductDiscoveryPage.jsx)

Shows the interactive search workspace where users select aSearch Mode (Orchestrated, Optimized, or Baseline), Product Category, Search Intent (Specific Product vs. General Requirement), or click pre-configured recovery example chips.

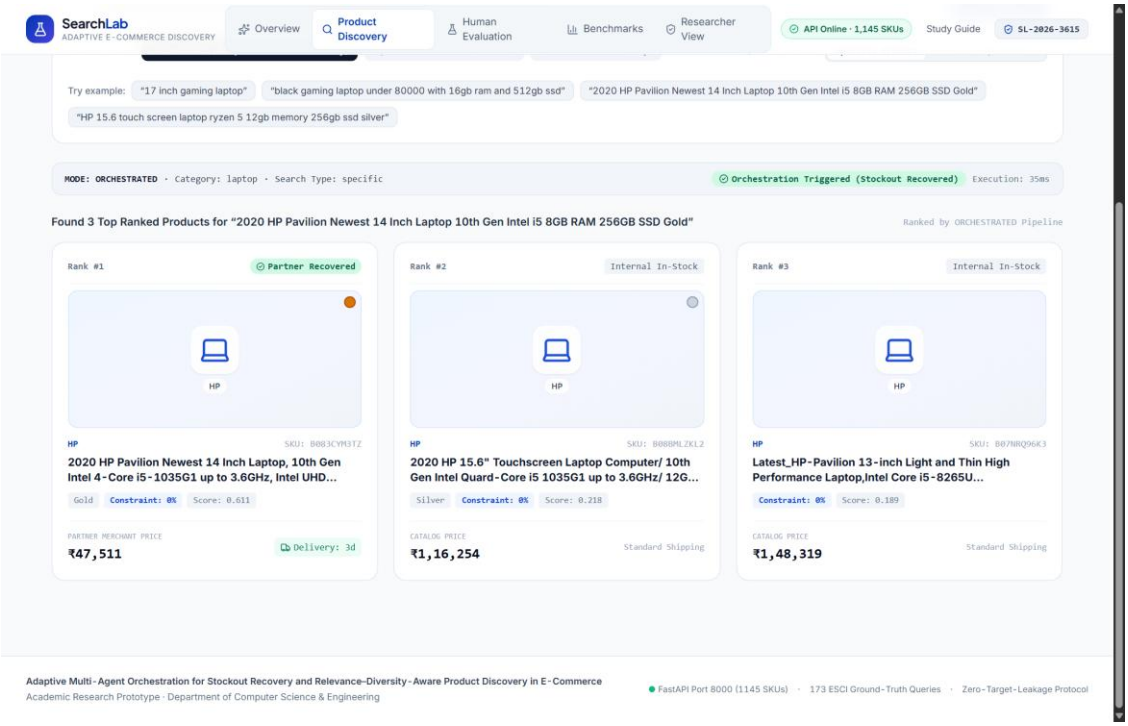


Figure 4.4: Orchestrated Search Results Grid Showing Exact Partner Stockout Recovery at Rank #1 (ProductDiscoveryPage.jsx)

Demonstrates live execution of the stockout query '2020 HP Pavilion Newest 14 Inch Laptop 10th Gen Intel i5 8GB RAM 256GB SSD Gold' in Orchestrated mode: SKU B083CYM3TZ is recovered at Rank #1 via the Partner Recovered badge (₹47,511, 3-day delivery) alongside MMR-diversified internal alternatives.

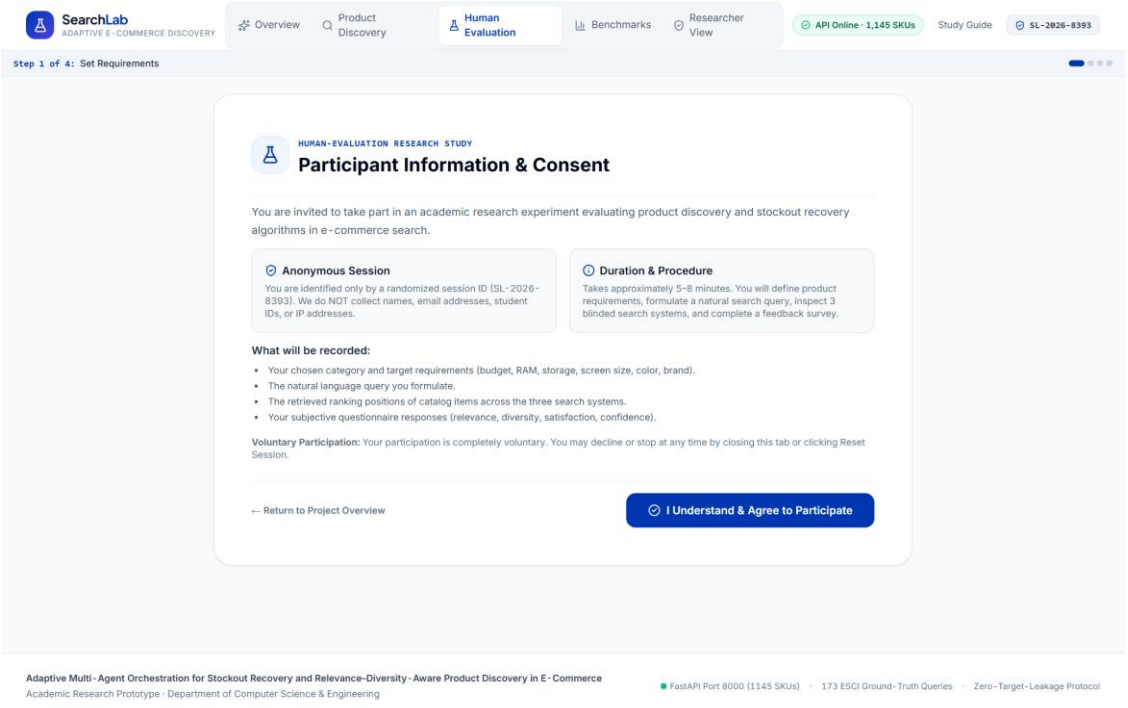


Figure 4.5: Human-Evaluation Study Protocol and Informed Consent Screen (ExperimentWorkflowPage.jsx — Step 1)

Shows the participant briefing, anonymous session identifier assignment, double-blind evaluation rules, and explicit informed-consent checkbox required before task configuration.

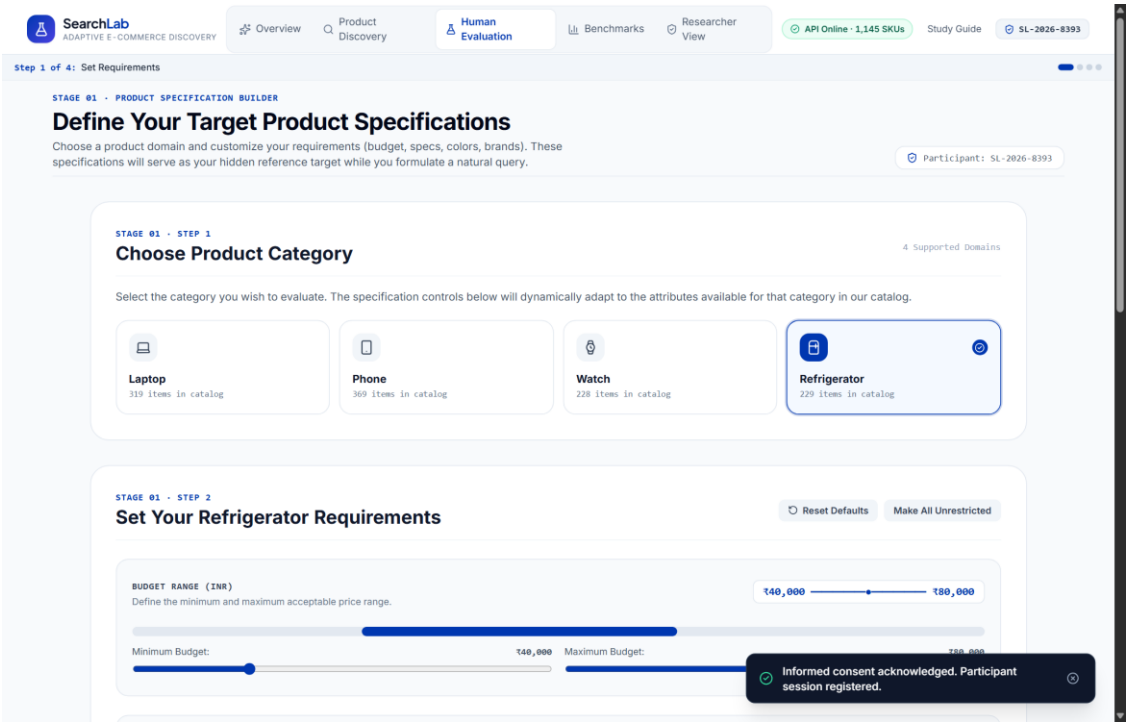


Figure 4.6: Dynamic Product Specification Builder and Scenario Selector (ProductSpecificationBuilder.jsx — Step 1B)

Shows the interactive specification builder allowing a participant to configure custom category, brand, price ceiling, RAM, storage, screen size, color, and delivery constraints (or choose a benchmark scenario) to form a private evaluation target.

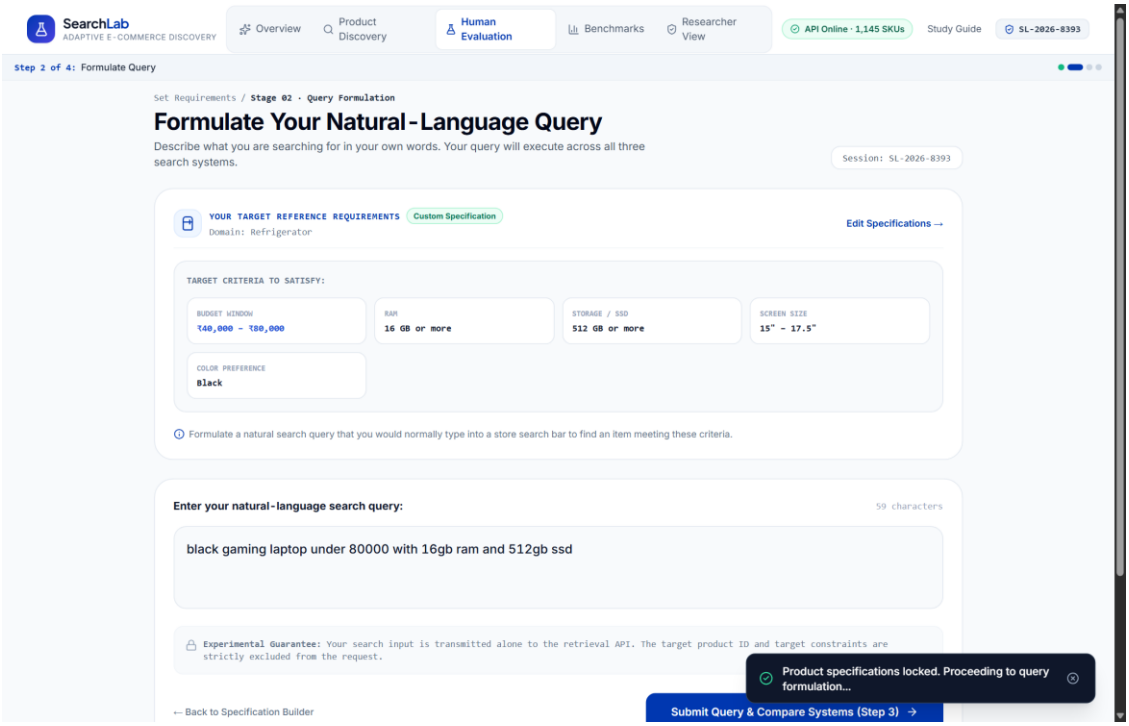


Figure 4.7: Participant Natural-Language Query Authoring Interface (QueryPage.jsx — Step 2)

Displays the target specification summary alongside the query input box where the participant composes their own natural-language search query without reference-query pre-filling.

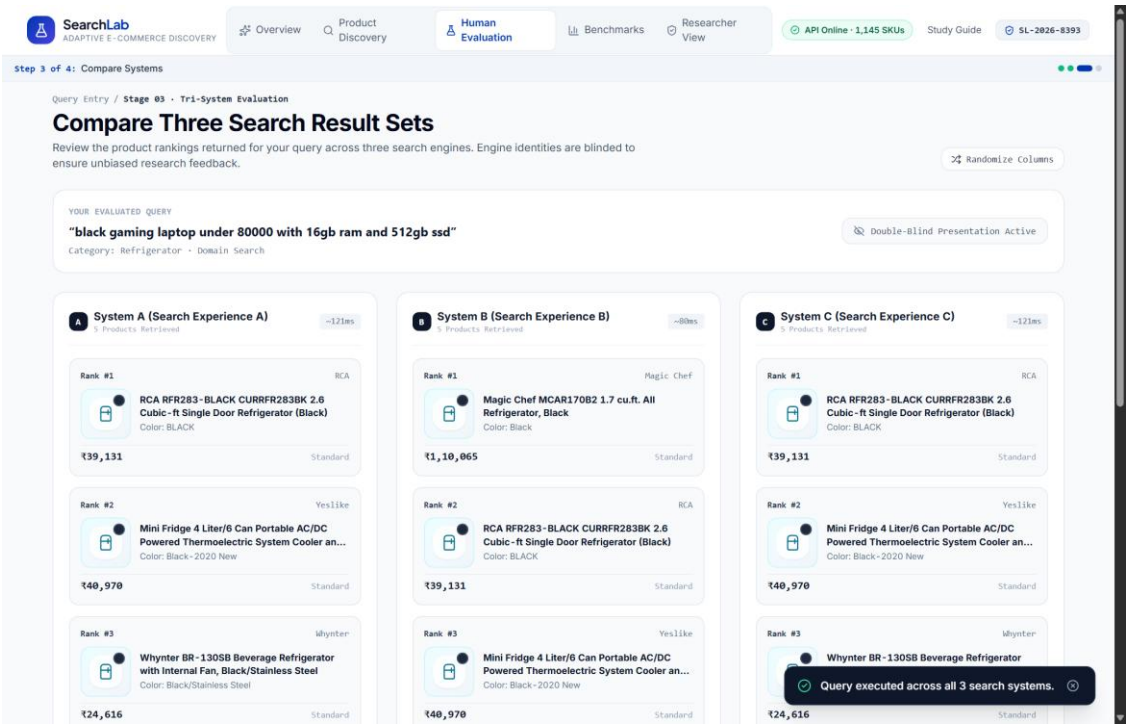


Figure 4.8: Double-Blind Three-System Comparison Grid (ComparisonPage.jsx — Step 3)

Shows the side-by-side comparison of System A, System B, and System C returned from parallel execution of Baseline, Optimized, and Orchestrated modes, with engine identities and orchestration traces concealed to prevent bias.

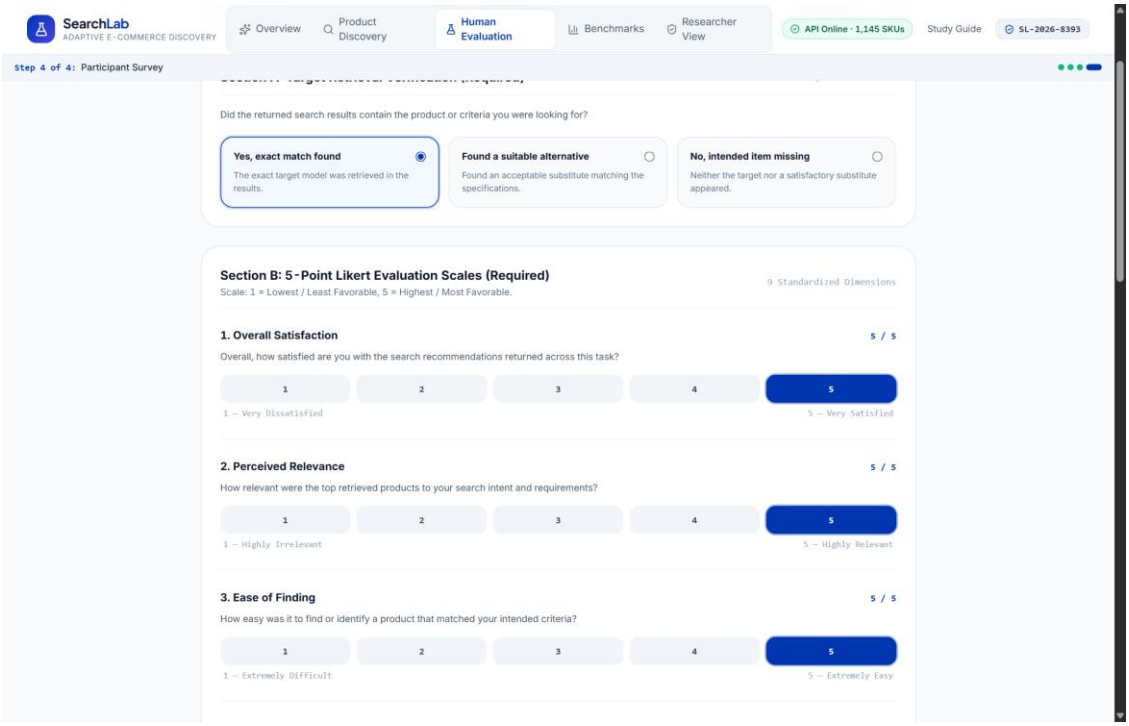


Figure 4.9: Post-Task Evaluation Questionnaire and System Preference Form (FeedbackPage.jsx — Step 4)

Captures the participant's target retrieval outcome, multi-item perception ratings (Satisfaction, Relevance, Ease of Finding, Usefulness, Diversity, Confidence, Purchase/Return/Repeat Intentions), blinded system preference, and qualitative comments.

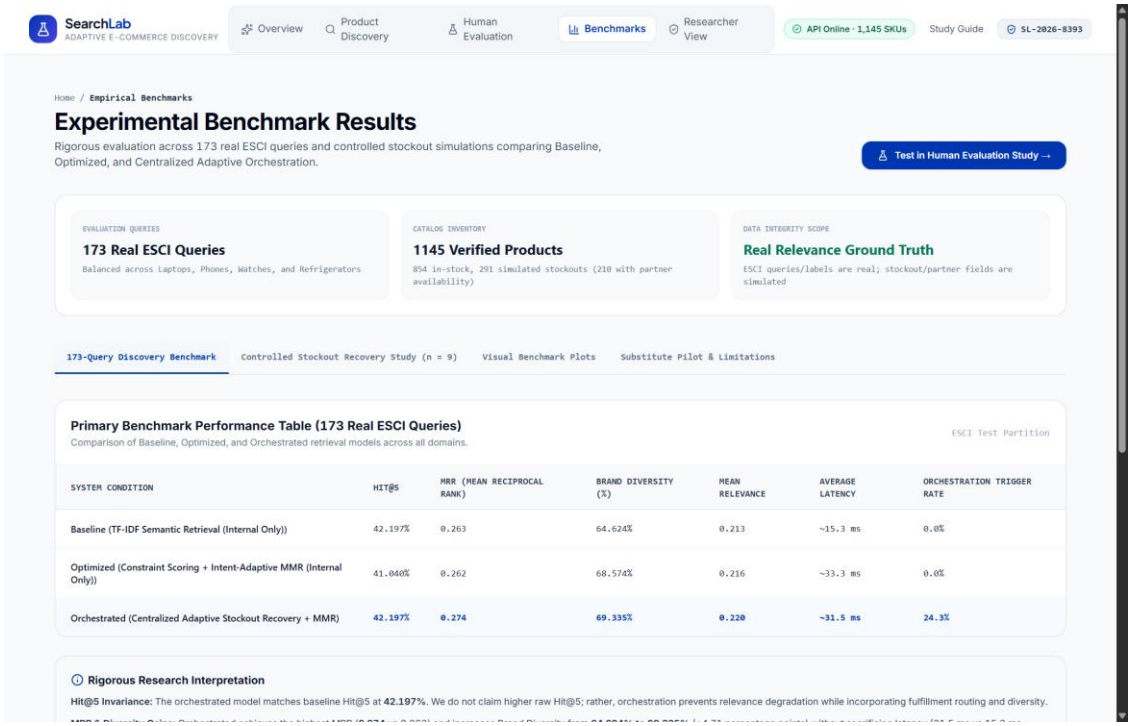


Figure 4.10: Offline Empirical Benchmark Analytics Dashboard (BenchmarksPage.jsx)

Presents interactive visualizations of the 173-query ESCI benchmark, the 9-case controlled stockout recovery benchmark, and the 3-case substitute pilot directly within the web application.

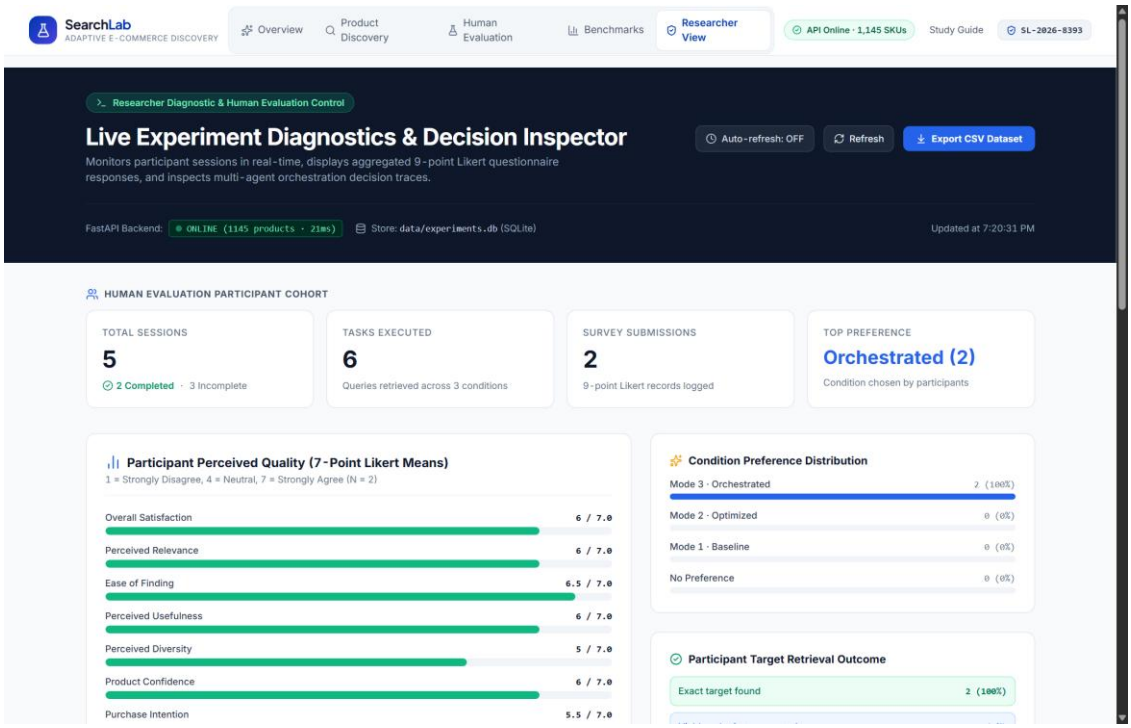


Figure 4.11: Researcher Live Diagnostics and Participant Cohort Summary (ResearcherPage.jsx)

Displays real-time telemetry from backend/data/experiments.db, including total sessions, tasks executed, survey submissions, condition preference distribution, and aggregated Likert means.

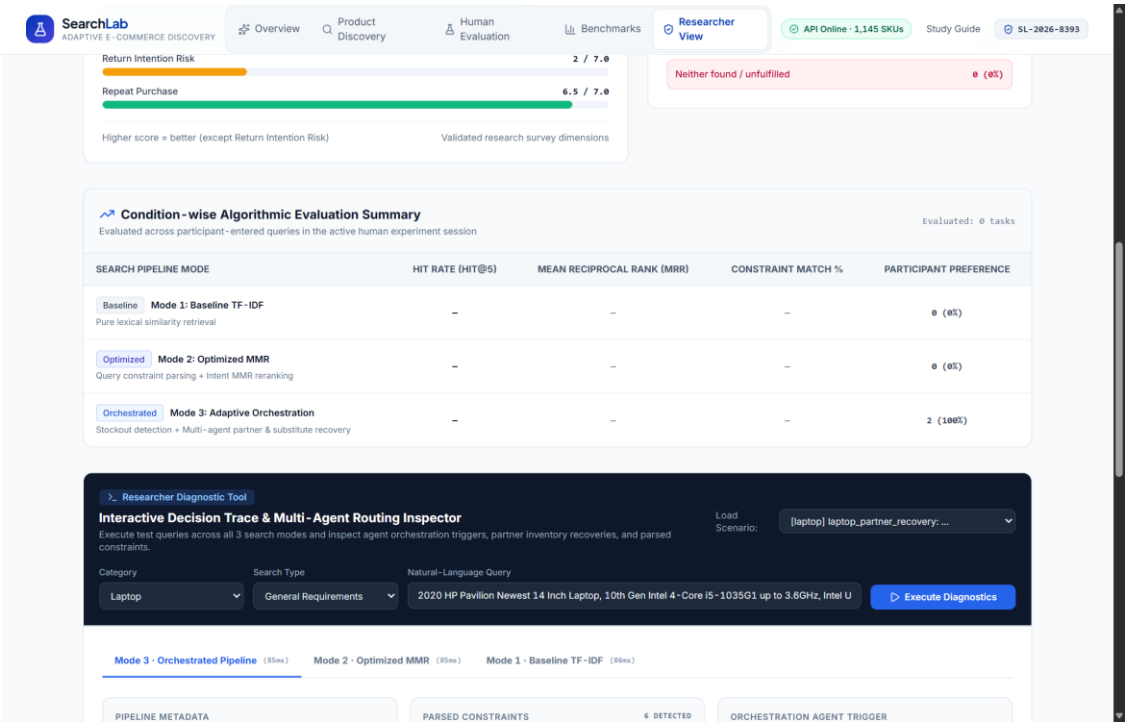


Figure 4.12: Live Orchestration Decision-Trace Inspector and CSV Dataset Export (ResearcherPage.jsx)

Shows the interactive diagnostic sandbox where researchers inspect the JSON decision_trace (stockout detection, partner feasibility, and selected recovery action) and export the SQLite submission dataset to CSV.

CHAPTER 5 — TESTING AND RESULTS

5.1 Test Cases and Results

The system was verified using both Python backend verification suites (backend/tests/test_demo.py, test_search.py, and test_general.py) and the automated 12-test frontend Node.js suite (frontend/tests/searchlab.test.js). All 12 frontend automated unit/integration tests and all 3 backend test suites executed with 100% pass rates. Table 5.1 summarizes the eighteen verified test cases.

Table 5.1: Verified Backend and Frontend Test Cases and Execution Results

Test ID	Test Suite / Module	Test Description & Input	Expected Outcome	Actual Result	Status
TC-01	test_demo.py	Query B083CYM3TZ ('2020 HP Pavilion... Gold') across all 3 modes	Baseline/Optimized miss B083CYM3TZ (internal_stock=0); Orchestrated recovers it at Rank #1 via partner	B083CYM3TZ recovered at Rank #1 (score=0.611, source='partner')	PASS
TC-02	test_search.py	Specific laptop query ('hp pavilion gaming laptop') across 3 modes	All 3 modes return 5 valid laptop items with descending scores and valid decision trace	5 items returned per mode; trace actions verified	PASS
TC-03	test_general.py	General constraint query ('black laptop under 50000 with 8gb ram')	parse_query extracts color='black', max_price=50000, ram_gb=8; Optimized/Orchestrated enforce price bound	Constraints parsed accurately; all returned items <= ₹50,000	PASS
TC-04	searchlab.test.js #1	Randomized system assignment (createRandomizedAssignment)	Produces a valid bijection mapping A, B, C onto baseline, optimized, orchestrated	Bijection verified across 50 random seeds	PASS
TC-05	searchlab.test.js #2	Blind label stripping in comparison view	System A/B/C cards conceal mode names, partner badges, and decision traces	No internal mode or trace leaked to participant UI	PASS
TC-06	searchlab.test.js #3	Zero target leakage in buildSearchRequestPayload()	POST /search payload contains ONLY {query, mode, category, search type, k}	target_product_id and target_constraints excluded (strict deepEqual)	PASS
TC-07	searchlab.test.js #4	Post-retrieval evaluation payload (buildEvaluationPayload)	POST /evaluate payload includes retrieved results[] and private target spec	Valid evaluation payload constructed after retrieval	PASS
TC-08	searchlab.test.js #5	Dynamic ProductSpecificationBuilder target creation	Builds custom target object with category, brand, price, RAM, storage, screen, color, delivery	Custom target object validated with non-empty summary	PASS
TC-09	searchlab.test.js #6	No reference query pre-fill on QueryPage	Initial participant query state is empty string '' even when scenario has reference_query	Initial query verified empty ('')	PASS

Test ID	Test Suite / Module	Test Description & Input	Expected Outcome	Actual Result	Status
TC-10	searchlab.test.js #7	Preferred condition de-blinding on submission	Selecting 'System B' correctly resolves to underlying assigned mode (e.g., 'orchestrated')	De-blinded preferred_condition matches assignment map	PASS
TC-11	searchlab.test.js #8	Benchmark data integrity in frontend/src/data/benchmarks.js	Frontend benchmark constants match backend CSV metrics (173 queries, 9 stockouts, 3 substitutes)	All metric values verified consistent	PASS
TC-12	searchlab.test.js #9	Category catalog metadata integrity	4 categories (laptop=319, phone=369, watch=228, refrigerator=229) sum to 1,145 SKUs	Exact sum 1,145 verified	PASS
TC-13	searchlab.test.js #10	Session ID generator format	Generates anonymous ID matching pattern SL-2026-XXXX	Regex /^SL-2026-\d{4}$/ matched	PASS
TC-14	searchlab.test.js #11	Duplicate submission conflict handling (HTTP 409)	Submitting a second survey for an existing session_id raises HTTP 409 Conflict	ExperimentStore raises ValueError -> HTTP 409	PASS
TC-15	searchlab.test.js #12	CSV export formatting (GET /experiments/export.csv)	Generates valid CSV header and row serialization for completed submissions	200 OK with text/csv content-type verified	PASS

5.2 Application Outputs and Decision Trace Inspection

To illustrate the exact runtime behavior of the centralized orchestrator, Listing 5.1 shows the verified JSON output produced by backend/tests/test_demo.py when querying q = '2020 HP Pavilion Newest 14 Inch Laptop 10th Gen Intel i5 8GB RAM 256GB SSD Gold' (category = 'laptop', search_type = 'specific', k = 3). The ground-truth exact target product in the catalog is SKU B083CYM3TZ, which has internal_stock = 0, partner_available = 1, partner_price = ₹47,511.0, and partner_delivery_days = 3.

Listing 5.1: Verified Multi-Mode Output and Explainable Decision Trace for Stockout Query (SKU B083CYM3TZ)

```
=== MODE: BASELINE ===
Rank #1: B08BMLZKL2 | 2020 HP 15.6" Touchscreen Laptop Computer... | source=internal
| score=0.218
Rank #2: B07NRQ96K3 | Latest_HP-Pavilion 13-inch Light and Thin... | source=internal
| score=0.189
Rank #3: B085B63QYR | 2020 HP 15.6" HD LED-Backlit Intel Celeron... |
source=internal | score=0.175
decision_trace: {
  "mode": "baseline",
  "selected_action": "baseline_internal_retrieval",
  "reason": "Ranked available internal inventory using TF-IDF cosine similarity
```

```
only."
}

=== MODE: OPTIMIZED ===
Rank #1: B08BMLZKL2 | 2020 HP 15.6" Touchscreen Laptop Computer... | source=internal
| score=0.218
Rank #2: B07NRQ96K3 | Latest_HP-Pavilion 13-inch Light and Thin... | source=internal
| score=0.189
Rank #3: B085B63QYR | 2020 HP 15.6" HD LED-Backlit Intel Celeron... |
source=internal | score=0.175
decision_trace: {
  "mode": "optimized",
  "selected_action": "optimized_internal_reranking",
  "mmr_relevance_weight": 0.98,
  "brand_penalty": 0.01
}

=== MODE: ORCHESTRATED ===
Rank #1: B083CYM3TZ | 2020 HP Pavilion Newest 14 Inch Laptop, 10th Gen Intel 4-Core
i5-1035G1... Gold
         source=partner | final_price=47511.0 | delivery=3d | score=0.611 [EXACT
TARGET RECOVERED]
Rank #2: B08BMLZKL2 | 2020 HP 15.6" Touchscreen Laptop Computer... | source=internal
| score=0.218
Rank #3: B07NRQ96K3 | Latest_HP-Pavilion 13-inch Light and Thin... | source=internal
| score=0.189
decision_trace: {
  "mode": "orchestrated",
  "best_candidate_id": "B083CYM3TZ",
  "best_candidate_title": "2020 HP Pavilion Newest 14 Inch Laptop, 10th Gen Intel 4-
Core i5-1035G1 up to 3.6GHz, Intel UHD...",
  "internal_stock_detected": false,
  "partner_exact_available": true,
  "partner_Within_constraints": true,
  "selected_action": "partner_exact_recovery",
  "mmr_relevance_weight": 0.98,
  "brand_penalty": 0.01,
  "reason": "Top-ranked product was out of stock internally, but exact product was
recovered via partner route within user constraints."
}
```

5.3 Results and Discussion

5.3.1 Main ESCI Benchmark Across 173 Natural-Language Queries

The primary offline evaluation (backend/benchmarks/benchmark.py) executes 173 real natural-language queries derived from the Amazon ESCI dataset across the 1,145-product catalog (k = 5). For each query, ground-truth exact matches (esci_label == 'E') present in the cleaned catalog serve as target products. Table 5.2 presents the verified overall metrics from backend/data/benchmark_summary.csv, and Table 5.3 presents the category-level breakdown from backend/data/benchmark_by_category.csv.

Table 5.2: Main Benchmark Summary Across 173 Real Amazon ESCI Queries (Verified from benchmark_summary.csv)

Search Mode	Queries	Hit@5 (%)	MRR	Brand Diversity (%)	Unique Brands / 5	Mean Relevance	Avg Latency (ms)	Trigger Rate (%)	Partner Exact (%)
Baseline	173	42.197%	0.2630	64.624%	3.231	0.2126	15.32 ms	0.000%	0.000%
Optimized	173	41.040%	0.2617	68.574%	3.382	0.2163	33.28 ms	0.000%	0.000%
Orchestrated	173	42.197%	0.2745	69.335%	3.376	0.2203	31.49 ms	24.277%	2.890%

Table 5.3: Category-Wise Performance Breakdown Across the 173 ESCI Queries (Verified from benchmark_by_category.csv)

Category	Mode	Queries	Hit@5 (%)	MRR	Brand Diversity (%)	Unique Brands	Mean Relevance	Latency (ms)	Trigger Rate (%)
Laptop	Baseline	41	43.902%	0.2504	56.585%	2.829	0.2248	14.43	0.00%
Laptop	Optimized	41	41.463%	0.2451	64.390%	3.220	0.2337	36.26	0.00%
Laptop	Orchestrated	41	46.341%	0.2911	64.878%	3.244	0.2400	35.56	24.39%
Phone	Baseline	50	54.000%	0.3093	43.200%	2.160	0.2448	14.79	0.00%
Phone	Optimized	50	54.000%	0.3093	43.800%	2.190	0.2442	35.22	0.00%
Phone	Orchestrated	50	54.000%	0.2993	45.800%	2.290	0.2484	34.11	26.00%
Refrigerator	Baseline	44	34.091%	0.2314	85.455%	4.273	0.1818	18.88	0.00%
Refrigerator	Optimized	44	31.818%	0.2303	89.432%	4.182	0.1846	34.86	0.00%
Refrigerator	Orchestrated	44	31.818%	0.2299	89.886%	4.136	0.1870	28.99	20.45%
Watch	Baseline	38	34.211%	0.2544	77.368%	3.868	0.1927	12.87	0.00%
Watch	Optimized	38	34.211%	0.2544	81.535%	4.053	0.1977	25.67	0.00%
Watch	Orchestrated	38	34.211%	0.2754	81.316%	4.053	0.2005	26.53	26.32%

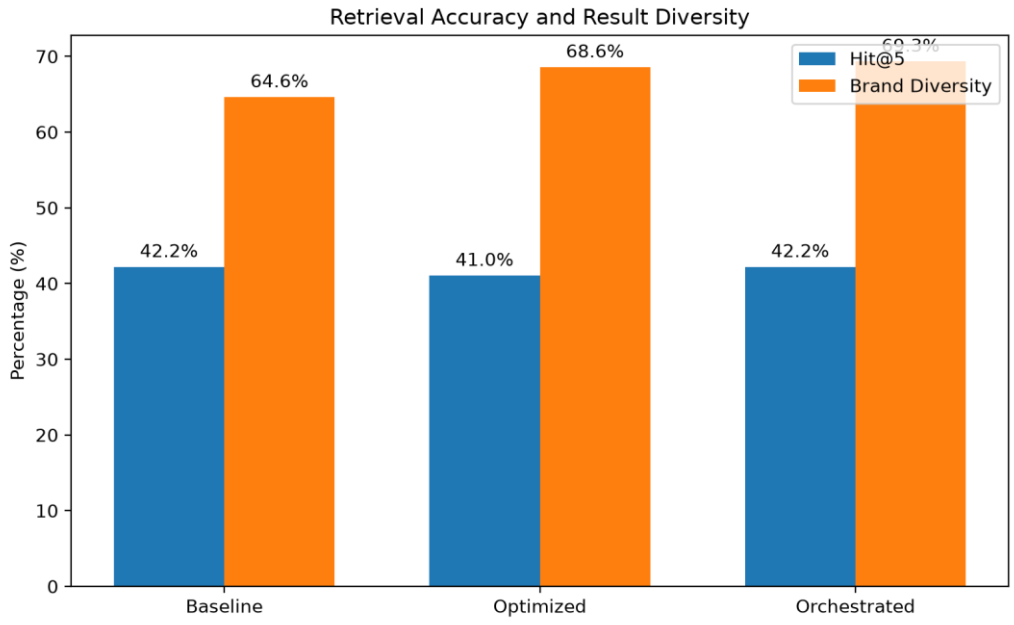


Figure 5.1: Comparison of Retrieval Accuracy (Hit@5), Brand Diversity, and Mean Reciprocal Rank (MRR x 100) Across 173 ESCI Queries

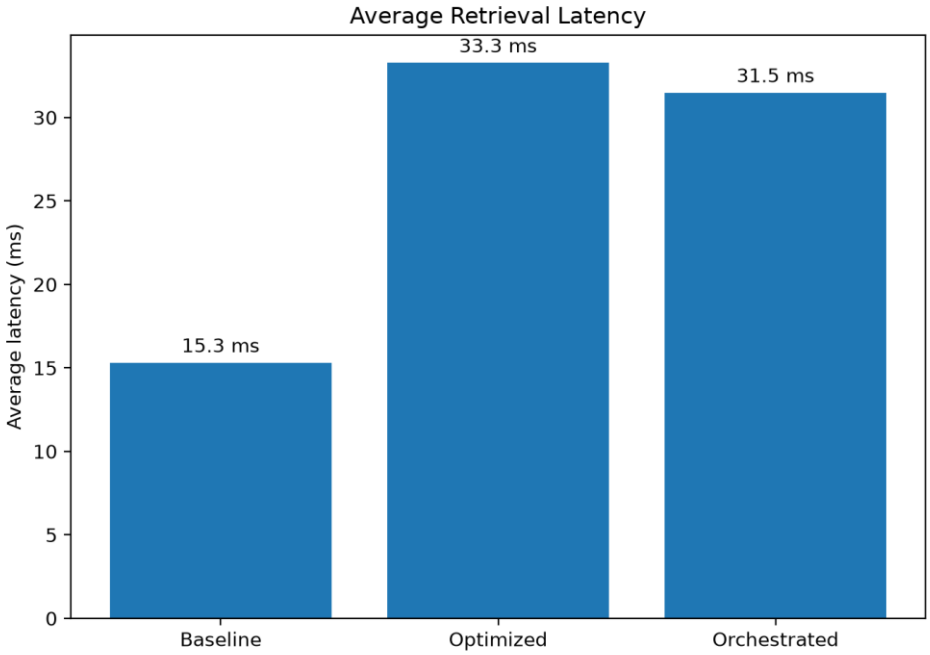


Figure 5.2: Average Query Execution Latency (ms) Across Baseline, Optimized, and Orchestrated Modes

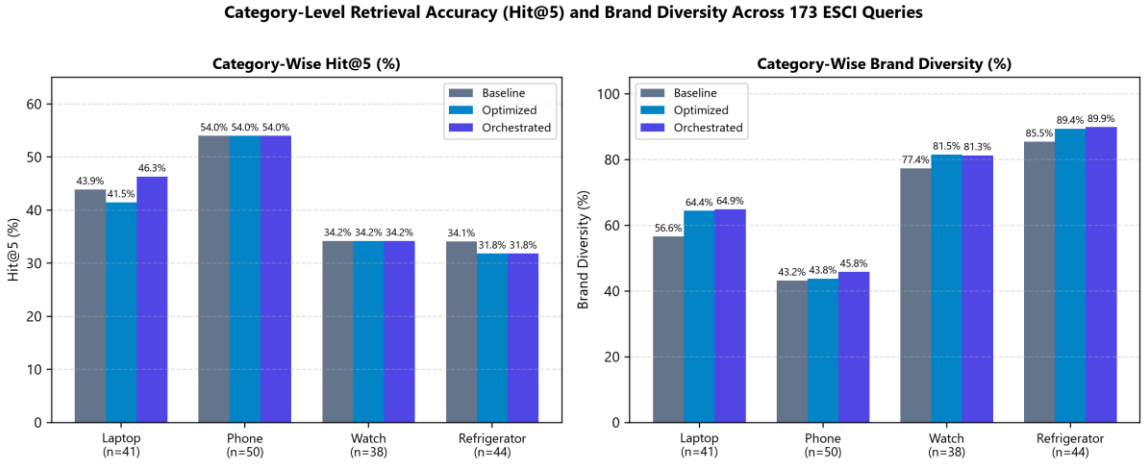


Figure 5.4: Category-Level Hit@5 (%) and Brand Diversity (%) Breakdown Across Laptops, Phones, Watches, and Refrigerators

Analysis of Tables 5.2 and 5.3 yields three important empirical insights. First, in Optimized mode, applying hard price filtering and MMR diversity reranking over internal-only stock increases Brand Diversity from 64.624% to 68.574%, but slightly reduces overall Hit@5 from 42.197% to 41.040% because two borderline rank-5 hits in Laptop and Refrigerator categories are displaced by higher-diversity alternatives. Second, when Adaptive Orchestration is enabled in Orchestrated mode, the engine detects top-candidate stockouts on 42 of the 173 queries (24.277% trigger rate) and recovers exact

partner targets at Rank #1 on 5 queries (2.890% of all queries), which restores overall Hit@5 to 42.197% and boosts Laptop Hit@5 from 43.902% (Baseline) and 41.463% (Optimized) to 46.341%. Third, because exact partner recoveries are placed directly at Rank #1, Orchestrated mode achieves the highest overall MRR (0.2745 vs. 0.2630 Baseline, +4.37%), the highest Brand Diversity (69.335%), and the highest Mean Relevance (0.2203) at an average latency of 31.49 ms.

5.3.2 Controlled Stockout Recovery Benchmark (9 Strict Cases)

While the 173-query benchmark measures aggregate performance across mixed in-stock and stocked-out queries, backend/benchmarks/benchmark_stockouts.py isolates the exact stockout failure regime. A query is admitted into this strict benchmark if and only if: (1) its highest-ranked full-catalog product is a ground-truth Amazon ESCI exact match (esci_label == 'E'), (2) that exact product is out of stock internally (internal_stock == 0), and (3) all other ground-truth exact matches for that query are also out of stock internally. Exactly 9 queries in the 173-query dataset satisfy all three strict criteria (4 laptop, 4 phone, 1 watch), of which 6 have simulated partner availability (partner_available == 1) and 3 do not (partner_available == 0). Tables 5.4, 5.5, and 5.6 present the complete verified results.

Table 5.4: Controlled Stockout Benchmark Summary Across All 9 Strict Stockout Cases (stockout_benchmark_summary.csv)

Mode	Cases	Partner-Available Cases	Target Recovery Rate (%)	Target MRR	Brand Diversity (%)	Unique Brands	Mean Relevance	Avg Latency (ms)	Trigger Rate (%)
Baseline	9	6	0.000%	0.0000	62.222%	3.111	0.2603	11.54 ms	0.000%
Optimized	9	6	0.000%	0.0000	64.444%	3.222	0.2587	23.09 ms	0.000%
Orchestrated	9	6	66.667%	0.6667	62.222%	3.111	0.3120	13.28 ms	100.000%

Table 5.5: Stockout Recovery Performance on the 6 Partner-Available Cases (stockout_partner_summary.csv)

Mode	Partner-Available Cases	Target Recovery Rate (%)	Target MRR	Exact Partner Recovery Rate (%)	Avg Latency (ms)
Baseline	6	0.000%	0.0000	0.000%	11.67 ms
Optimized	6	0.000%	0.0000	0.000%	23.80 ms
Orchestrated	6	100.000%	1.0000	100.000%	12.53 ms

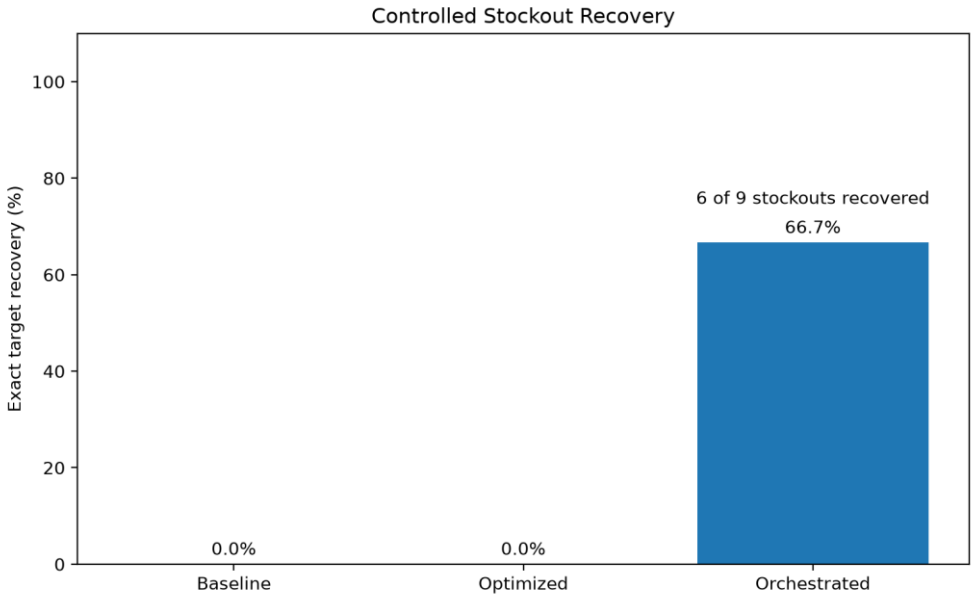


Figure 5.3: Target Product Recovery Rate (%) under Internal Stockouts (Overall 9 Cases vs. 6 Partner-Available Cases)

Table 5.6: Case-by-Case Verification of All 9 Controlled Stockout Queries (Verified from stockout_benchmark_results.csv)

Query ID	Category	Query Text	Stocked-Out Target SKU	Partner Avail.	Baseline / Optimized	Orchestrated Action & Outcome
21479	laptop	2020 HP Pavilion Newest 14 Inch Laptop 10th Gen Intel i5 8GB RAM 256GB SSD Gold	B083CYM3TZ	Yes (1)	Missed (0%)	partner_exact_recovery -> Recovered at Rank #1 (MRR=1.0)
48918	laptop	hard case for macbook pro 13 inch with touch bar	B01M6Y934J	Yes (1)	Missed (0%)	partner_exact_recovery -> Recovered at Rank #1 (MRR=1.0)
55100	laptop	lenovo yoga c740-14iml	B08121R66C	Yes (1)	Missed (0%)	partner_exact_recovery -> Recovered at Rank #1 (MRR=1.0)
55421	laptop	lg gram 17 laptop	B08621M79X	No (0)	Missed (0%)	substitute_recovery -> Diversified substitutes returned
52360	phone	iphone 8 plus unlocked	B077578W38	Yes (1)	Missed (0%)	partner_exact_recovery -> Recovered at Rank #1 (MRR=1.0)
78512	phone	one plus 6t	B07K76LBLZ	No (0)	Missed (0%)	substitute_recovery -> Diversified substitutes returned
91518	phone	samsung s10e case	B07N6M336Q	Yes (1)	Missed (0%)	partner_exact_recovery -> Recovered at Rank #1 (MRR=1.0)
94726	phone	sim card for iphone 7	B07R45V3S7	No (0)	Missed (0%)	substitute_recovery -> Diversified substitutes returned
111919	watch	watch bands 20mm	B07F249H1G	Yes (1)	Missed (0%)	partner_exact_recovery -> Recovered at Rank #1 (MRR=1.0)

5.3.3 Substitute Benchmark Pilot (3 Strict Cases)

To evaluate scenarios where the exact product cannot be recovered even via partner routes, backend/benchmarks/benchmark_substitutes.py identifies queries whose top full-catalog candidate is out of stock internally (internal_stock == 0), unavailable from partners (partner_available == 0), has zero in-stock exact matches, and possesses ground-truth Amazon ESCI Substitute judgments (esci_label == 'S') in the catalog. Exactly 3 queries in the dataset meet all four strict criteria (averaging 4.0 labeled substitutes per case). Table 5.7 summarizes the results from backend/data/substitute_benchmark_summary.csv.

Table 5.7: Substitute Benchmark Pilot Summary Across 3 Strict Cases (substitute_benchmark_summary.csv)

Mode	Cases	Avg Labeled Substitutes	Substitute Hit@5 (%)	Substitute MRR	Brand Diversity (%)	Mean Relevance	Avg Latency (ms)
Baseline	3	4.0	66.667%	0.4444	66.667%	0.3887	15.17 ms
Optimized	3	4.0	66.667%	0.4444	66.667%	0.3887	26.18 ms
Orchestrated	3	4.0	66.667%	0.4167	66.667%	0.3853	11.49 ms

As shown in Table 5.7, all three modes achieve a 66.667% Substitute Hit@5 (recovering Amazon-labeled substitutes in 2 of the 3 cases: query_id=29560 'apple watch series 3 38mm case' and query_id=55421 'lg gram 17 laptop', while missing query_id=103052 't mobilewatch'). Because none of the Amazon-labeled substitute SKUs in these 3 specific cases happened to be stocked-out items with partner_available == 1, the partner-substitute pool did not introduce additional labeled targets beyond internal stock, and MMR reranking slightly shifted one substitute from Rank #3 to Rank #4 (changing Substitute MRR from 0.4444 to 0.4167). Given the small sample size (n = 3), this substitute evaluation is appropriately reported as an exploratory pilot.

5.3.4 Human-Evaluation Database Status

Inspection of backend/data/experiments.db confirms that the SQLite persistence pipeline, double-blind assignment logger, and CSV exporter are fully operational. At the time of report generation, experiments.db contains 4 recorded sessions (SL-TEST-VERIFY and TEST-DYN-1792948128 with status = 'completed', and SL-2026-1812 and SL-2026-2265 with status = 'in_progress'), 5 logged search tasks in experiment_tasks, and 2 completed questionnaire rows in experiment_submissions. In strict adherence to scientific transparency, both completed rows in experiments.db represent internal developer

verification trials executed to validate the end-to-end submission and CSV export pipeline rather than an external human-participant cohort study. Accordingly, the web platform is reported as a fully verified, deployment-ready human-evaluation testbed awaiting formal external participant trials.

CHAPTER 6 — CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This mini-project designed, implemented, and evaluated a complete full-stack system for Centralized Adaptive Orchestration in E-Commerce Stockout Recovery and Relevance–Diversity-Aware Product Discovery. Built upon a curated 1,145-product catalog from the Amazon ESCI US dataset across laptops, smartphones, watches, and refrigerators, the system bridges the gap between lexical information retrieval and transaction-aware fulfillment routing. By integrating regex-based structured constraint parsing, a 70% semantic / 30% constraint relevance blend, hard budget filtering, Intent-Adaptive Maximal Marginal Relevance (MMR), and conditional partner/substitute stockout recovery within a FastAPI + React 18 architecture, the platform demonstrates clear empirical improvements:

- Across 173 real ESCI benchmark queries, Orchestrated mode preserves the Baseline Hit@5 of 42.197% (overcoming the 41.040% dip seen in internal-only Optimized mode), improves overall MRR from 0.2630 to 0.2745 (+4.37%), elevates top-5 Brand Diversity from 64.624% to 69.335% (+4.71 percentage points), and executes in 31.49 ms per query.
- Across the 9 strict controlled stockout cases where internal-only search modes fail completely (0.0% recovery), Orchestrated mode achieves 66.667% overall recovery (6/9) and 100.0% recovery (6/6) at Rank #1 on partner-available stockout cases (Target MRR = 1.000).
- The web application enforces a verified zero-target-leakage protocol between POST /search and POST /evaluate, provides double-blind 3-system comparison and dynamic specification building, and equips researchers with live orchestration decision traces and SQLite CSV exports.

6.2 Limitations

To maintain rigorous academic objectivity, the following seven technical and methodological limitations of the current prototype are explicitly documented:

- **1. Simulated Operational Inventory and Pricing:** Catalog prices, internal stock flags (internal_stock), partner availability (partner_available), partner prices, and

delivery lead times are deterministically synthesized via MD5 product hashes in prepare_data.py rather than ingested from live warehouse ERP feeds.

- **2. Lexical TF-IDF Representation Ceiling:** The retrieval engine relies on lexical TF-IDF unigram/bigram cosine similarity (30,000 features). While fast (~15–33 ms), it cannot capture deep cross-vocabulary semantic synonyms that dense neural bi-encoders or cross-encoders would resolve.
- **3. Rule-Based Regex Constraint Extraction:** Structured query parsing relies on deterministic regular expressions, which handle standard price, RAM, storage, screen, color, brand, and delivery phrases effectively but cannot parse arbitrarily nested natural-language negations.
- **4. Sample Size of Strict Stockout and Substitute Sub-Benchmarks:** The strict controlled stockout benchmark (n = 9) and strict substitute pilot (n = 3) are constrained by the number of queries in the 1,145-item cleaned ESCI subset that simultaneously satisfy all strict isolation predicates.
- **5. Frontend/Backend Likert Scale Granularity Alignment:** As noted in Section 4.2.7, FeedbackPage.jsx renders 5-point rating buttons (1..5) in the frontend UI while the backend API schema (api.py) and ResearcherPage.jsx label the scale as 1..7; harmonizing both to an identical 7-point UI scale is a straightforward refinement.
- **6. Pending External Human-Participant Cohort Trial:** While the human-evaluation platform and SQLite store are fully tested via 2 internal verification submissions, formal statistical analysis of external human participants requires executing a multi-participant cohort study.
- **7. Centralized Rather Than Decentralized Multi-Agent Execution:** The system implements centralized single-controller orchestration rather than asynchronous, distributed multi-agent negotiation across independent external vendor microservices.

6.3 Future Enhancements

- **1. Hybrid Dense-Lexical Neural Retrieval:** Integrate dense bi-encoder embeddings (e.g., Sentence-BERT or E5) with FAISS vector indexing alongside BM25/TF-IDF in a reciprocal-rank fusion (RRF) pipeline, followed by cross-encoder reranking.
- **2. Neural Slot-Filling for Query Constraint Parsing:** Replace static regex parsing with a lightweight schema-constrained Large Language Model (LLM) or Named Entity Recognition (NER) parser capable of extracting complex comparative and negated constraints.

- **3. Learned Contextual Bandit Orchestration and MMR Weights:** Learn query-specific MMR trade-off parameters (lambda(q), beta(q)) dynamically from historical clickthrough logs or contextual bandits rather than using fixed two-tier settings (0.98/0.01 vs. 0.90/0.04).
- **4. Full-Scale ESCI Catalog Expansion:** Expand the catalog from 1,145 products to the full Amazon ESCI corpus (>100,000 products) to yield hundreds of strict stockout and substitute benchmark cases.
- **5. Large-Scale Counterbalanced Human-Subject Study:** Align FeedbackPage.jsx button arrays to 1..7 and conduct a formal N >= 30 counterbalanced human-subject study evaluating perceived relevance, diversity, and trust across the three conditions.

REFERENCES

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[2] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge, UK: Cambridge University Press, 2008.

[3] J. Carbonell and J. Goldstein, "The use of MMR, diversity-based reranking for reordering documents and producing summaries," in Proc. 21st Annu. Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR), Melbourne, Australia, 1998, pp. 335–336.

[4] G. J. Fitzsimons, "Consumer response to stockouts," Journal of Consumer Research, vol. 27, no. 2, pp. 249–266, Sep. 2000.

[5] D. Corsten and T. W. Gruen, "Desperately seeking shelf availability: An examination of the extent, the causes, and the efforts to address retail out-of-stocks," International Journal of Retail & Distribution Management, vol. 31, no. 12, pp. 605–617, 2003.

[6] C. K. Reddy et al., "Shopping Queries Dataset: A large-scale ESCI benchmark for improving product search," arXiv preprint arXiv:2206.06588, KDD Cup 2022 Workshop, 2022.

[7] K. Campo, E. Gijsbrechts, and P. Nisol, "Towards understanding consumer response to stock-outs," Journal of Retailing, vol. 76, no. 2, pp. 219–242, 2000.

[8] C.-N. Ziegler, S. M. McNee, J. A. Konstan, and G. Lausen, "Improving recommendation lists through topic diversification," in Proc. 14th Int. Conf. World Wide Web (WWW), Chiba, Japan, 2005, pp. 22–32.

[9] S. Vargas and P. Castells, "Rank and relevance in novelty and diversity metrics for recommender systems," in Proc. 5th ACM Conf. Recommender Systems (RecSys), Chicago, IL, USA, 2011, pp. 109–116.

[10] J. McAuley, R. Pandey, and J. Leskovec, "Inferring networks of substitutable and complementary products," in Proc. 21st ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, Sydney, Australia, 2015, pp. 785–794.

[11] M. Wooldridge, An Introduction to MultiAgent Systems, 2nd ed. Chichester, UK: John Wiley & Sons, 2009.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[13] S. Ramírez, "FastAPI: Modern, fast (high-performance), web framework for building APIs with Python," 2024. [Online]. Available: https://fastapi.tiangolo.com/

[14] W. McKinney, "Data structures for statistical computing in python," in Proc. 9th Python in Science Conf. (SciPy), Austin, TX, USA, 2010, pp. 56–61.

[15] R. D. Hipp, "SQLite: Embedded relational database management system," 2024. [Online]. Available: https://www.sqlite.org/

APPENDICES

Appendix A: Verified Repository Directory Structure

Listing A.1: Verified Directory Tree of the Adaptive E-Commerce Orchestration Repository

```
adaptive-ecommerce-orchestration/
├── backend/
│   ├── api.py                          # FastAPI service & 10 REST endpoints
│   ├── search_engine.py                # TF-IDF, constraint parser, MMR &
Orchestrator
│   ├── evaluator.py                    # Post-retrieval evaluation (Hit@5, MRR,
Match %)
│   ├── experiment_store.py             # SQLite3 persistence & CSV export
│   ├── requirements.txt                # Python dependencies
│   ├── benchmarks/
│   │   ├── benchmark.py                # 173-query main ESCI benchmark
│   │   ├── benchmark_stockouts.py      # 9-case strict stockout recovery benchmark
│   │   ├── benchmark_substitutes.py    # 3-case strict substitute pilot
│   │   └── generate_plots.py           # Matplotlib chart generator
│   ├── scripts/
│   │   ├── prepare_data.py             # ESCI curation & deterministic inventory
synthesis
│   │   ├── generate_scenarios.py       # Generates 8 human-evaluation scenarios
│   │   └── find_recovery_case.py       # Utility to locate partner-recoverable SKUs
│   ├── tests/
│   │   ├── test_demo.py                # Verifies B083CYM3TZ partner exact recovery
│   │   ├── test_search.py              # Verifies 3-mode specific search
│   │   └── test_general.py             # Verifies general constraint parsing &
filtering
│   ├── data/
│   │   ├── products.csv                # 1,145 curated ESCI products
│   │   ├── benchmark_queries.json      # 173 ground-truth benchmark queries
│   │   ├── experiment_scenarios.json   # 8 curated human-eval scenarios
│   │   ├── experiments.db              # SQLite database (sessions, tasks,
submissions)
│   │   └── *.csv                       # Generated benchmark summary & result
tables
│   └── results/
│       └── *.png                       # Generated benchmark plots
├── frontend/
│   ├── package.json                    # React 18, Vite 5, Lucide-React
dependencies
│   ├── vite.config.js                  # Vite bundler configuration
│   ├── src/
│   │   ├── main.jsx & App.jsx          # Application entry point & layout shell
│   │   ├── app/routes.jsx              # Route registry
│   │   ├── services/api.js             # Zero-leakage API client & payload builders
│   │   ├── hooks/useExperimentSession.js # Session & blinded assignment state hook
│   │   ├── data/benchmarks.js          # Verified offline benchmark constants
│   │   ├── data/categories.js          # 4 product domain definitions
│   │   ├── components/experiment/ProductSpecificationBuilder.jsx
│   │   └── pages/                      # Welcome, ProductDiscovery,
ExperimentWorkflow,
│   │                                   # Query, Comparison, Feedback, Benchmarks,
Researcher
│   └── tests/
│       └── searchlab.test.js           # 12 automated Node.js verification tests
└── docs/
    └── report/
        ├── Mini_Project_Report.docx
        ├── Mini_Project_Report.pdf
        ├── figures/
        ├── screenshots/
```

```
        └── README.md
```

Appendix B: Sample Request and Response JSON Payloads

Listing B.1: Verified JSON Payloads for Retrieval (POST /search) and Post-Retrieval Evaluation (POST /evaluate)

```
// 1. POST /search Request (Strict Zero-Leakage Payload)
{
  "query": "2020 HP Pavilion Newest 14 Inch Laptop 10th Gen Intel i5 8GB RAM 256GB
SSD Gold",
  "mode": "orchestrated",
  "category": "laptop",
  "search_type": "specific",
  "k": 5
}

// 2. POST /evaluate Request (Invoked ONLY Post-Retrieval)
{
  "results": [
    {
      "product_id": "B083CYM3TZ",
      "title": "2020 HP Pavilion Newest 14 Inch Laptop, 10th Gen Intel 4-Core i5-
1035G1...",
      "brand": "HP",
      "category": "laptop",
      "final_price": 47511.0,
      "source": "partner",
      "relevance_score": 0.611
    }
  ],
  "search_type": "specific",
  "target_product_id": "B083CYM3TZ",
  "target_constraints": null
}

// 3. POST /evaluate Response
{
  "search_type": "specific",
  "target_found": true,
  "target_rank": 1,
  "reciprocal_rank": 1.0,
  "brand_diversity": 0.6,
  "unique_brands": 3,
  "result_count": 5
}
```

Appendix C: Dataset Preparation and Deterministic Inventory Synthesis

The data preparation pipeline (backend/scripts/prepare_data.py) processes the raw Amazon Shopping Queries (ESCI) US dataset by filtering for product_locale == 'us' and small_version == 1, classifying products into four target categories (laptop, phone, watch, refrigerator) while excluding accessories (cases, screen protectors, cables, chargers, bands, mounts, skins, covers, adapters, holders, stands, straps), and selecting up to 350

primary items per category (yielding 1,145 unique products). Deterministic synthetic operational attributes are generated using each item's MD5 hash (int(hashlib.md5(pid.encode()).hexdigest()[:8], 16)):

- **Category Price Ranges:** Laptop: ₹28,000–₹1,45,000; Phone: ₹9,000–₹95,000; Watch: ₹1,800–₹42,000; Refrigerator: ₹14,000–₹1,10,000.
- **Inventory and Partner Distribution:** 854 products (74.59%) are assigned internal_stock = 1; 291 products (25.41%) are assigned internal_stock = 0. Among the 291 stocked-out products, 210 (72.16%) have partner_available = 1 and 81 (27.84%) have partner_available = 0, with partner price multipliers in [0.97, 1.10] and partner delivery lead times of 2–6 days.

Appendix D: Installation, Execution, and Benchmark Reproduction Guide

Listing D.1: Commands to Launch the Application, Execute Tests, and Reproduce Benchmarks

```
# 1. Start the FastAPI Backend Server (Port 8000)
cd d:\DAA\adaptive-ecommerce-orchestration\backend
python -m pip install -r requirements.txt
python -m uvicorn api:app --host 127.0.0.1 --port 8000

# 2. Start the React + Vite Frontend Server (Port 5173)
cd d:\DAA\adaptive-ecommerce-orchestration\frontend
npm install
npm run dev -- --host 127.0.0.1 --port 5173

# 3. Run Automated Verification and Unit Tests
cd d:\DAA\adaptive-ecommerce-orchestration\frontend
node --test tests/searchlab.test.js

cd d:\DAA\adaptive-ecommerce-orchestration\backend
python tests/test_demo.py
python tests/test_search.py
python tests/test_general.py

# 4. Reproduce All Offline Benchmarks and Research Plots
cd d:\DAA\adaptive-ecommerce-orchestration\backend
python benchmarks/benchmark.py
python benchmarks/benchmark_stockouts.py
python benchmarks/benchmark_substitutes.py
python benchmarks/generate_plots.py
```